University of Nottingham
UK | CHINA | MALAYSIA

COMP4139 Machine Learning (MLE)

Deep Convolutional Neural Networks - Part 2

Dr Shreyank N Gowda
Dr Direnc Pekaslan

School of Computer Science
University of Nottingham

Recap

- What is a Convolutional Neural Network?
- What is a convolution?
- What are the problems with it?
- Formula for output with stride and padding = (N-F+2P)/S+1
- How do we handle color images with neural networks?
- What is vanishing gradient?

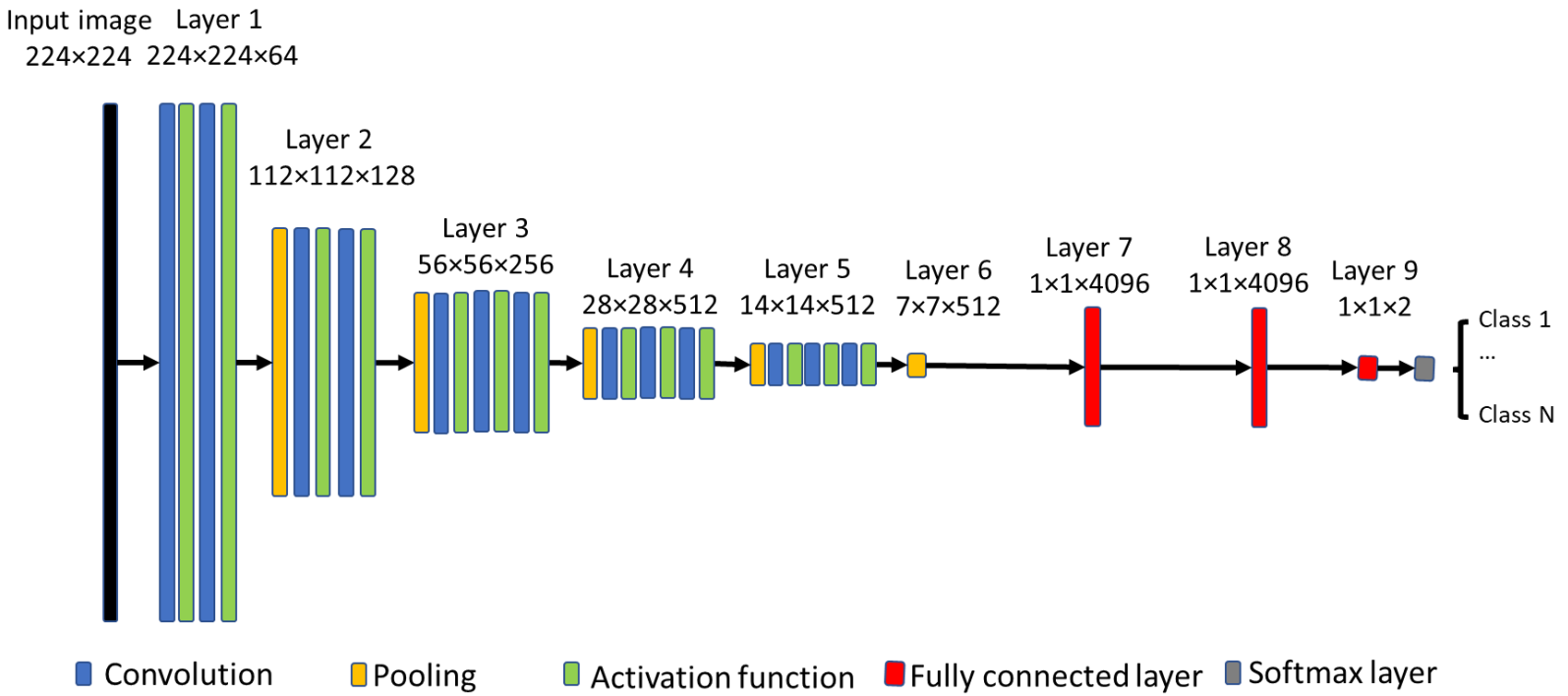
Learning Objectives

- Pooling
- Number of parameters in a CNN
- Why convolution?
- Stochastic and mini-batch gradient descent
- Exponential Averaging
- Momentum
- Adam

Basic Components of Convolutional Neural Network

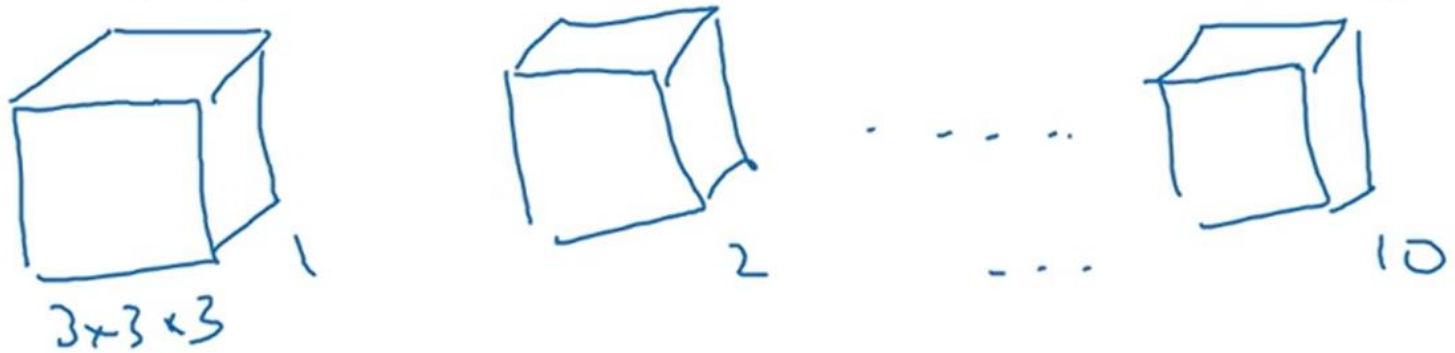
- Convolution operator
- Pooling
- Activation function
- Fully connected layer
- Loss functions
- Optimisation methods

Example Deep CNN for classification



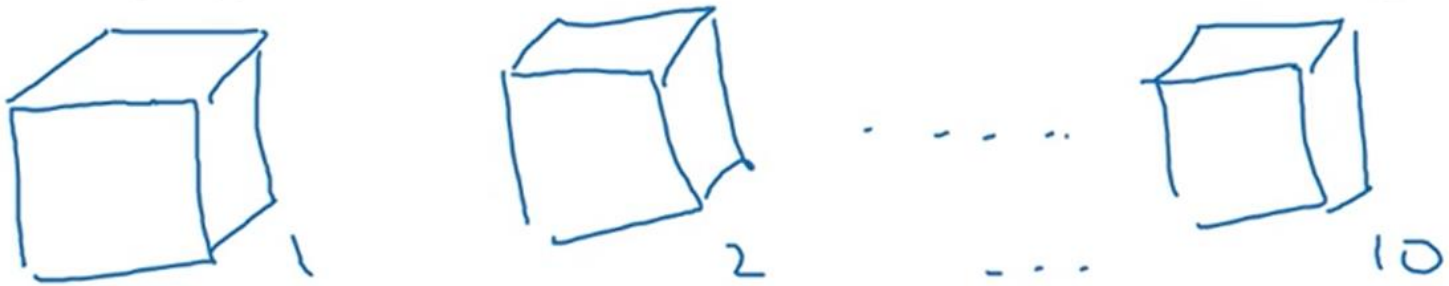
Number of parameters

If you have 10 filters that are 3 x 3 x 3 in one layer of a neural network, how many parameters does that layer have?



Number of parameters

If you have 10 filters that are 3 x 3 x 3 in one layer of a neural network, how many parameters does that layer have?



3×3×3

27 parameters.

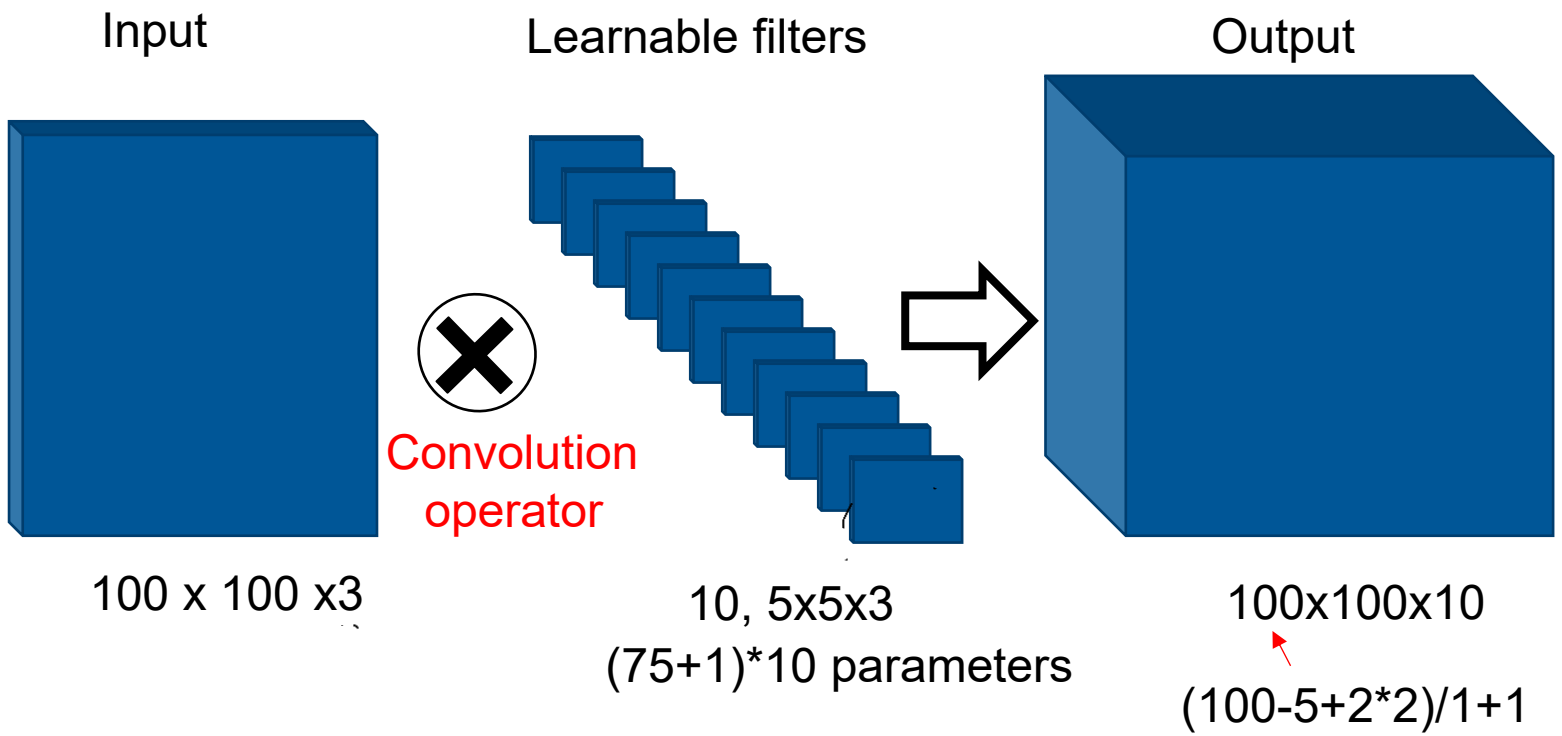
+bias

→ 28 parameters.

280 parameters..

Convolution Operator

- Convolve the input image with a set of learnable small size filters
- Input size(N); filter size (F); zero padding (P); stride (S);
- Parameters to be optimised; output feature map size calculation (N-F+2P)/S+1;



Why convolution operator?

- Parameter sharing

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

$*$

1	0	-1
1	0	-1
1	0	-1

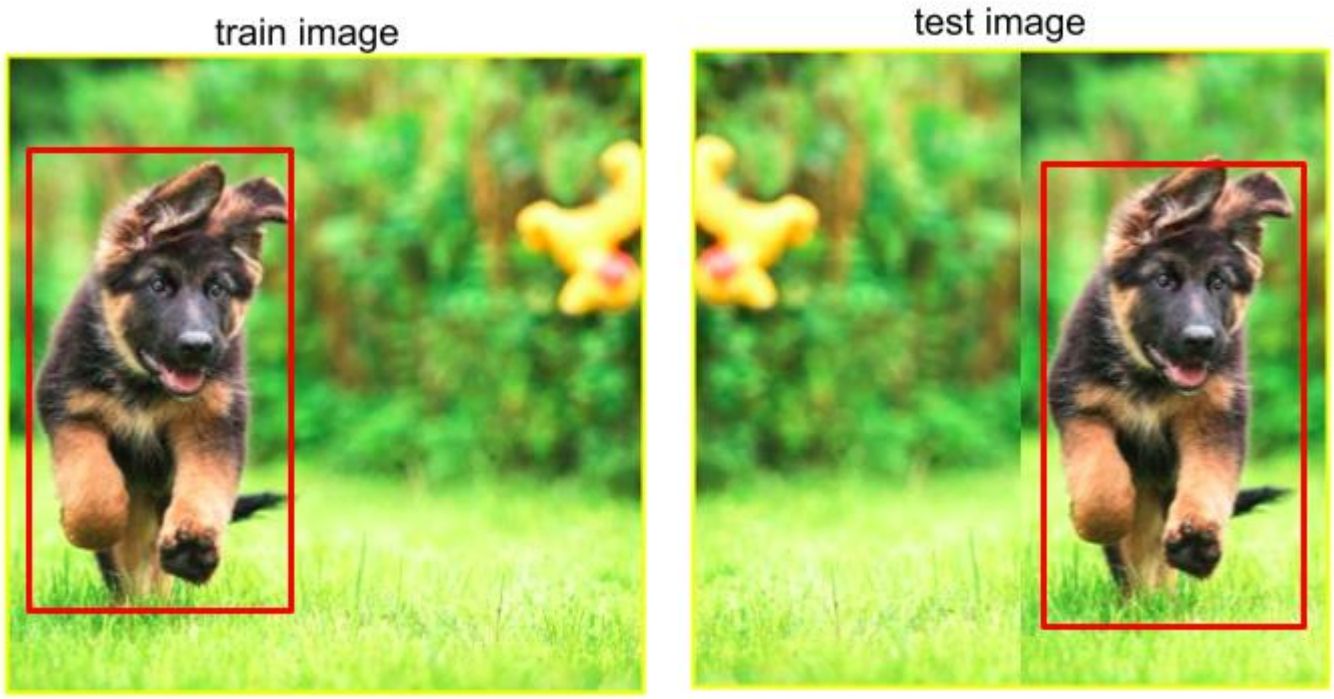
$=$

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

Parameter sharing: A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

Why convolution operator?

- Parameter sharing
- Spatial hierarchy
- Translation invariance

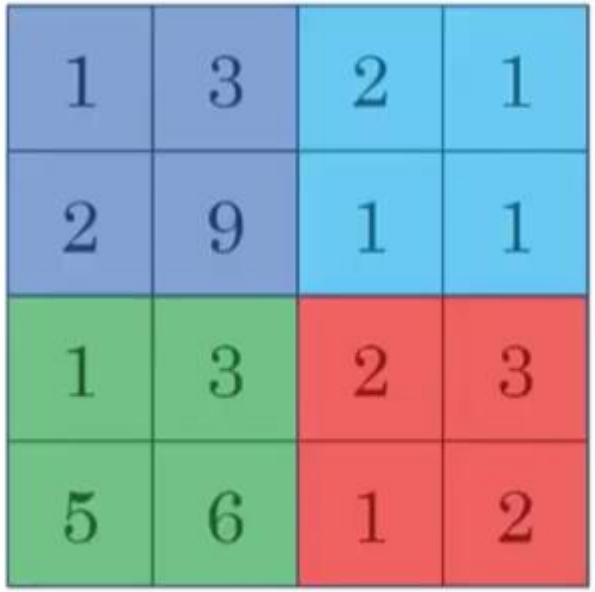


Pooling

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

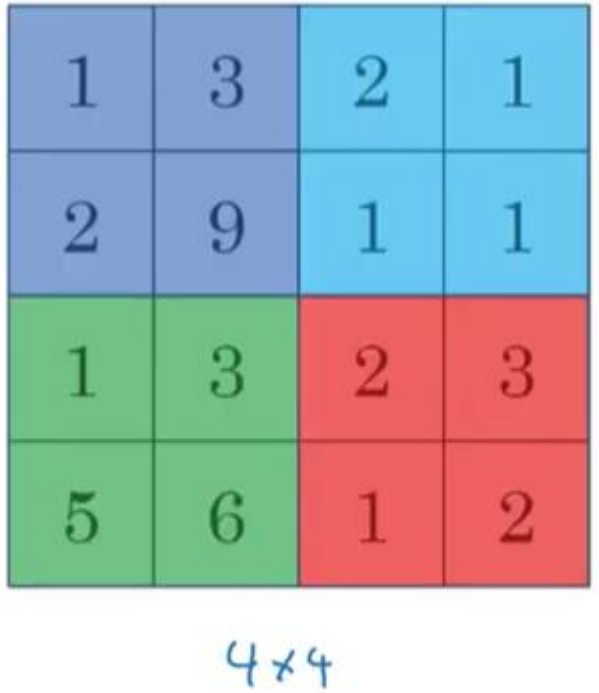
4×4

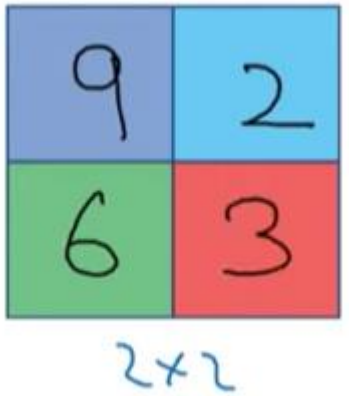
Pooling



4×4

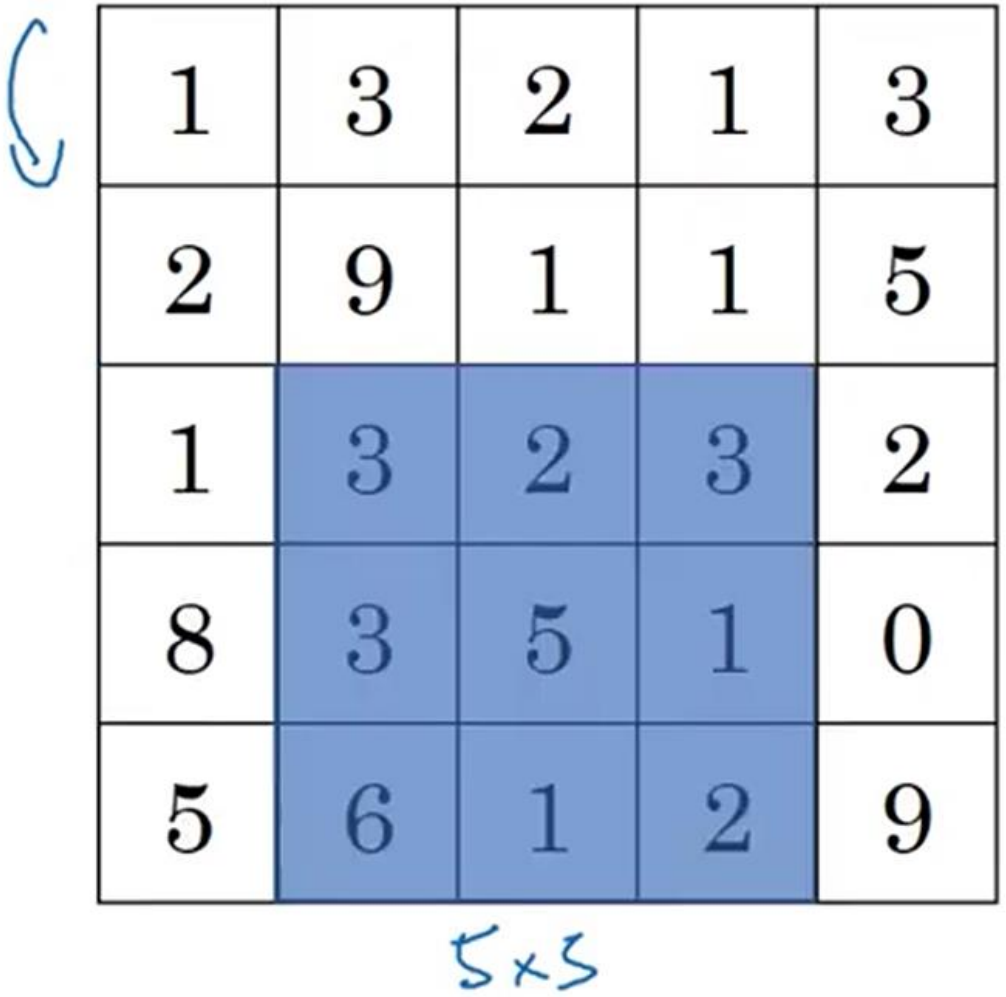
Pooling

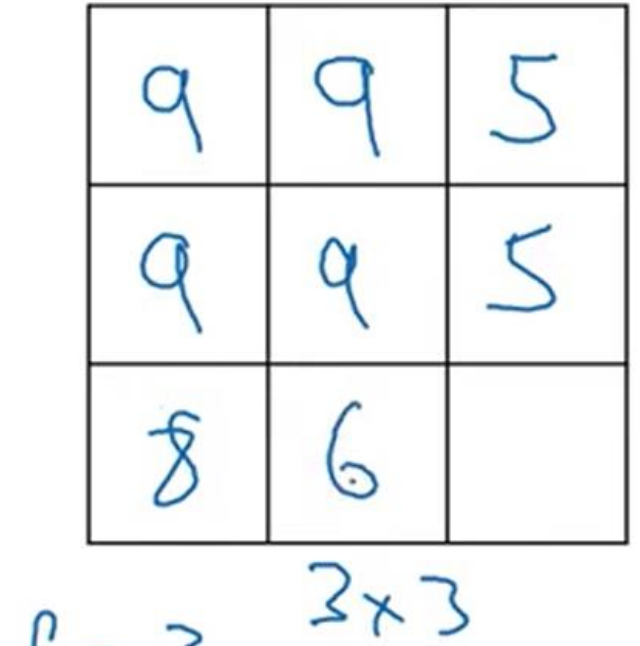




Hyperparameters
f = 2, s = 2
No parameters!

Max Pooling





$f = 3$

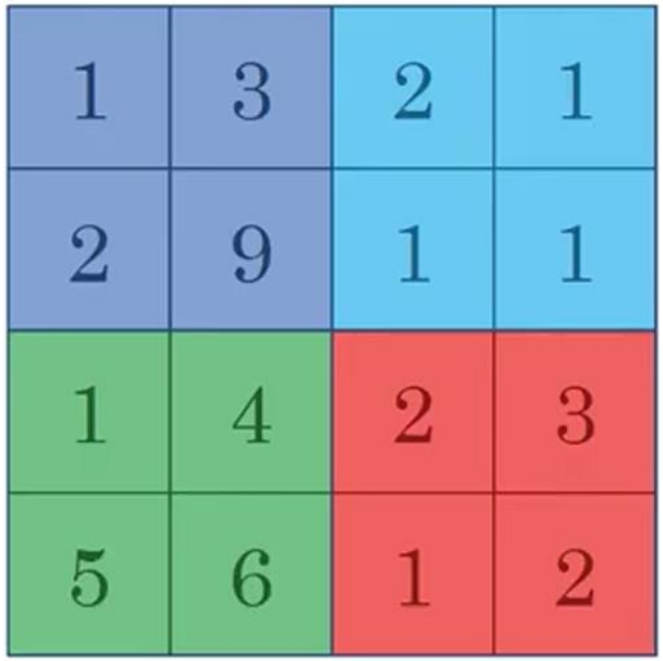
$s = 1$

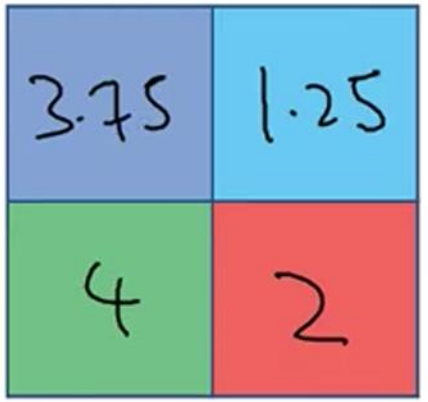
$$\left[\frac{n + 2p - f}{s} + 1\right]$$

Why max pooling?

- Feature preservation
- Reducing size of feature map
- Slowly going out of fashion. Generative models do not use, can replace with larger strides etc

Average Pooling





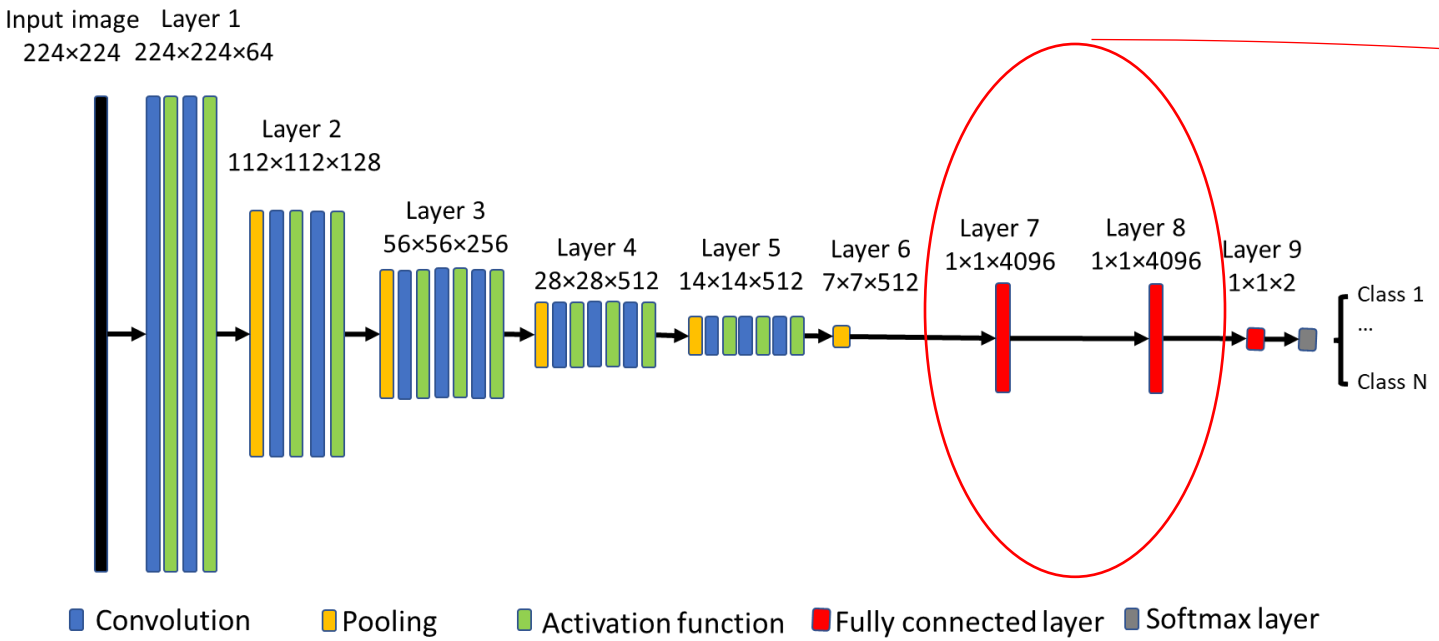
$f = 2$

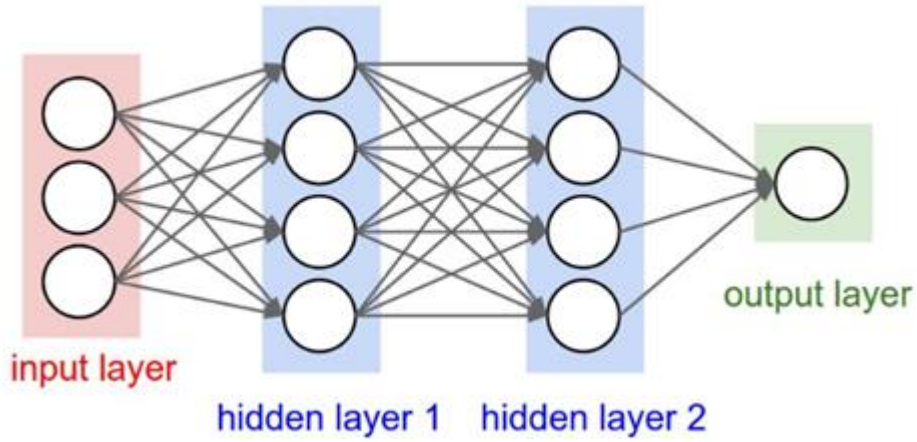
$s = 2$

Not used as it can blur or smooth out features

Fully Connected Layer and Output Activation

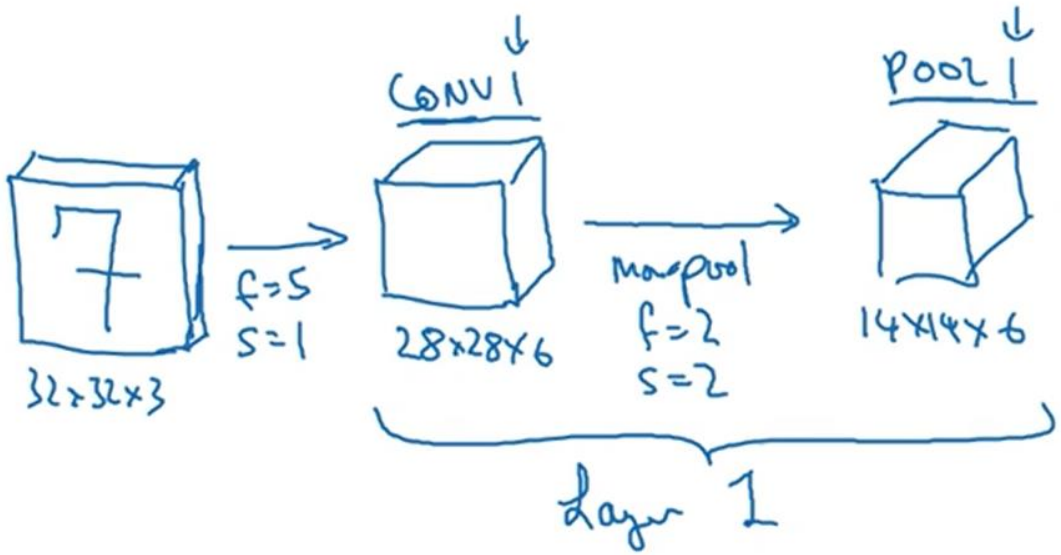
- Perform global feature learning: fully connected to all activations in the previous layer, as the same in MLP.
- Output activation: Softmax for classification, Linear for regression



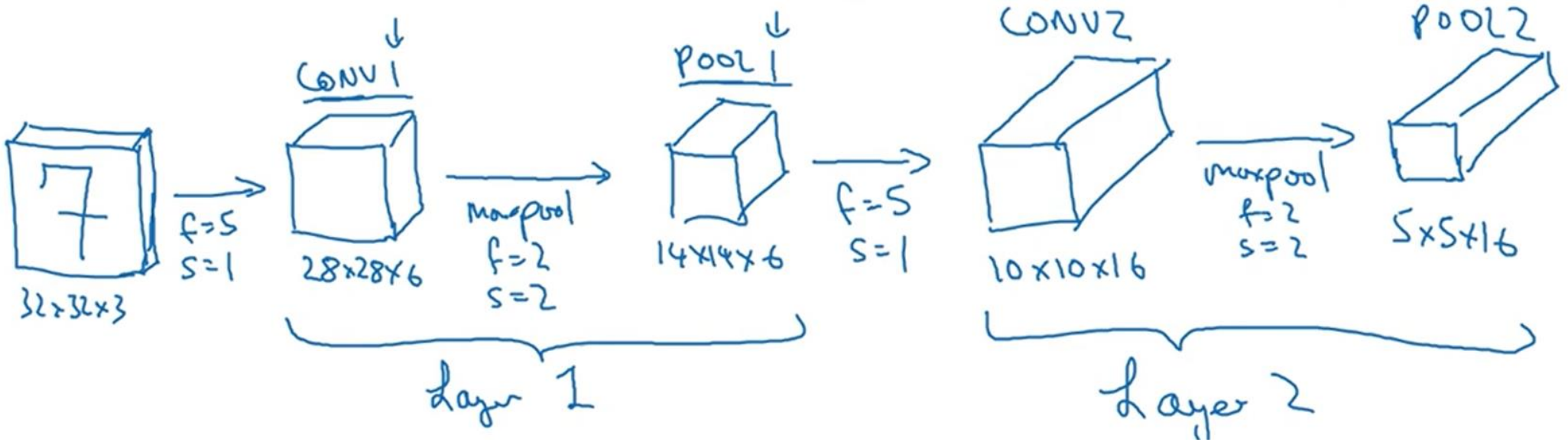


Multilayer Perceptron (MLP) Network

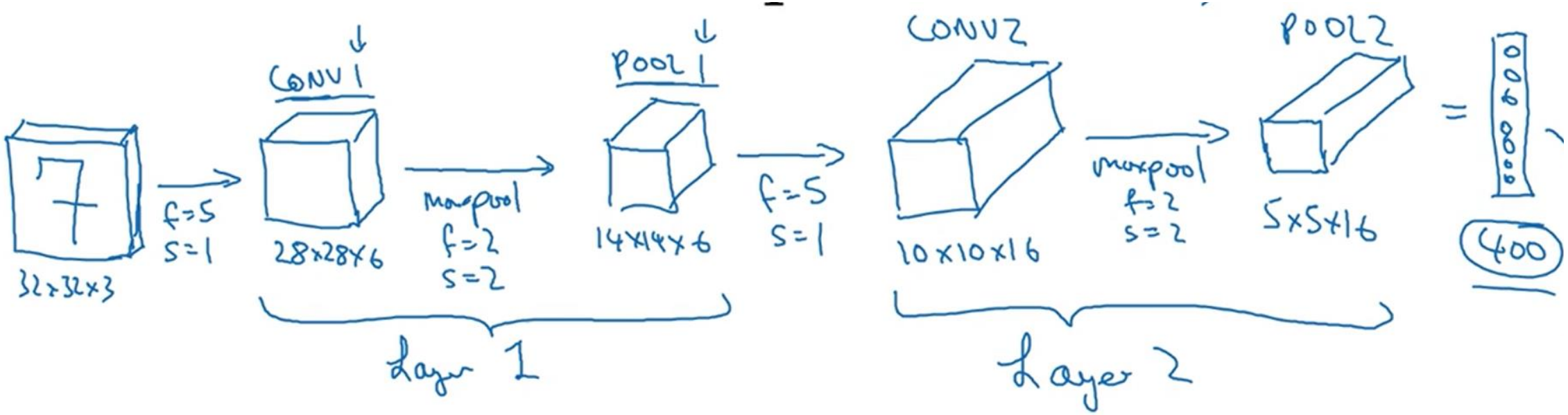
Full CNN Le-Net 5



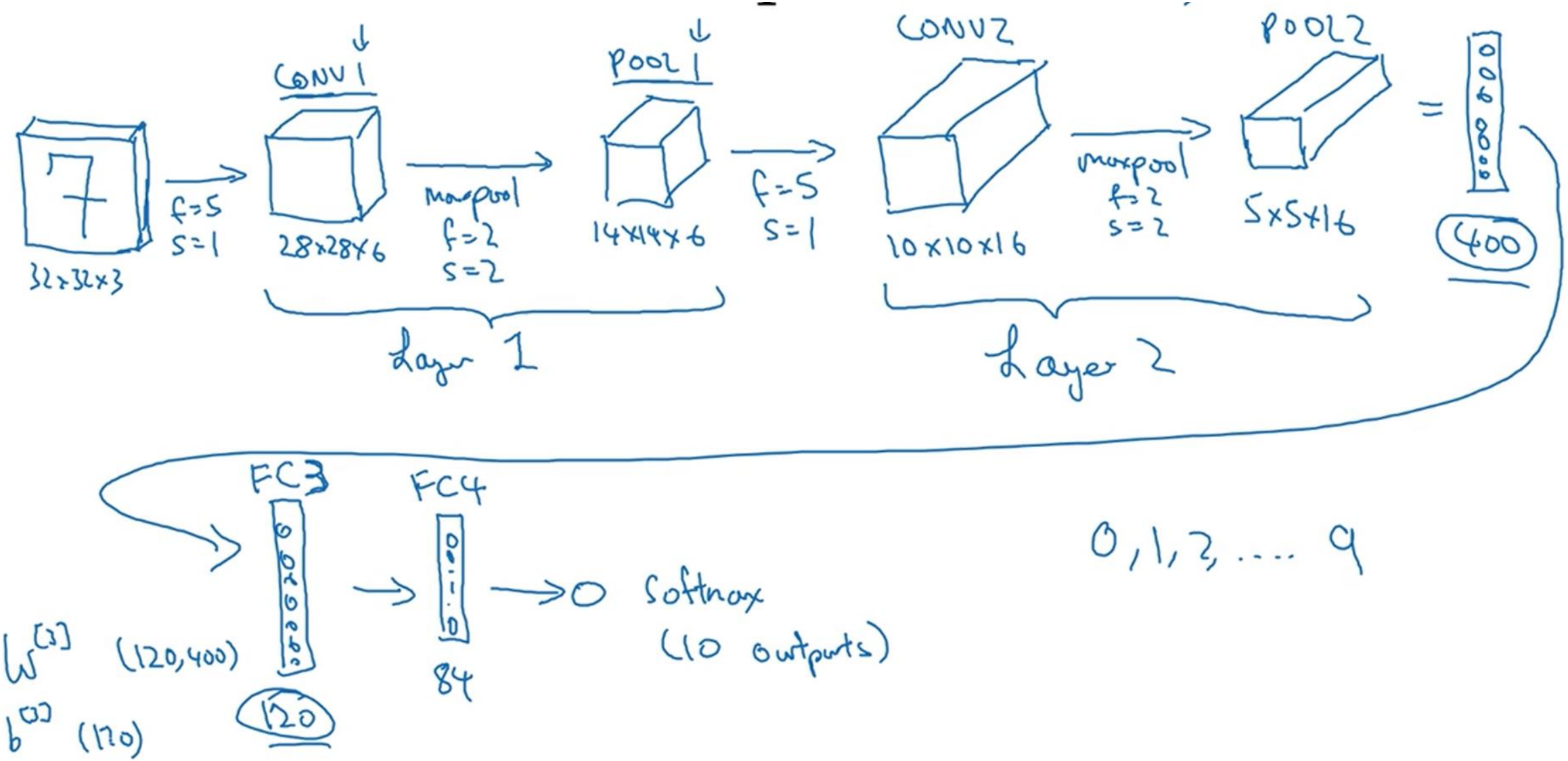
Full CNN Le-Net 5



Full CNN Le-Net 5



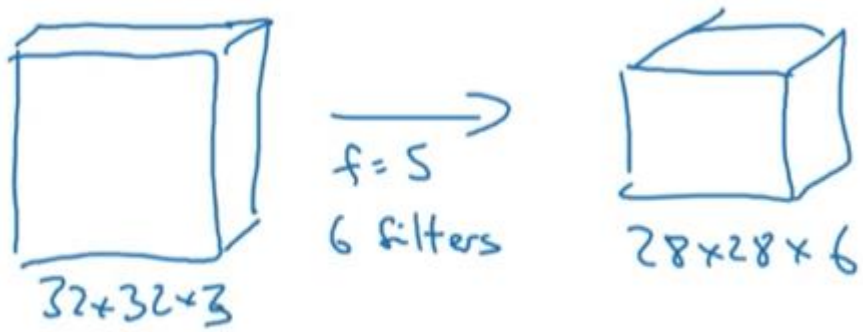
Full CNN Le-Net 5



Number of parameters in the CNN

	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072 $a^{[0]}$	0
CONV1 (f=5, s=1)	(28,28,8)	6,272	208
POOL1	(14,14,8)	1,568	0
CONV2 (f=5, s=1)	(10,10,16)	1,600	416
POOL2	(5,5,16)	400	0
FC3	(120,1)	120	48,001
FC4	(84,1)	84	10,081
Softmax	(10,1)	10	841

Comparing CNN vs ANN



3,072

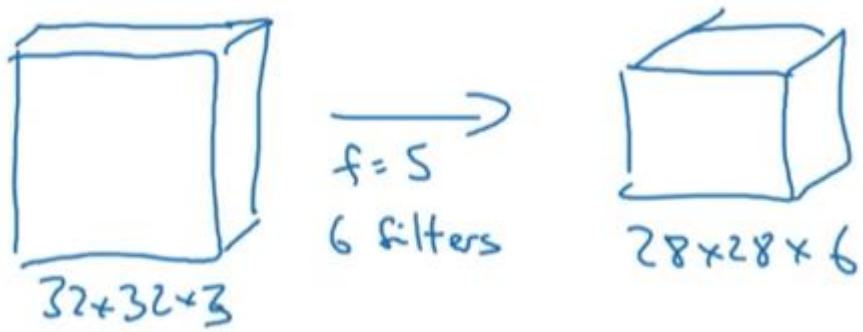
4,704

5x5 − 25

26

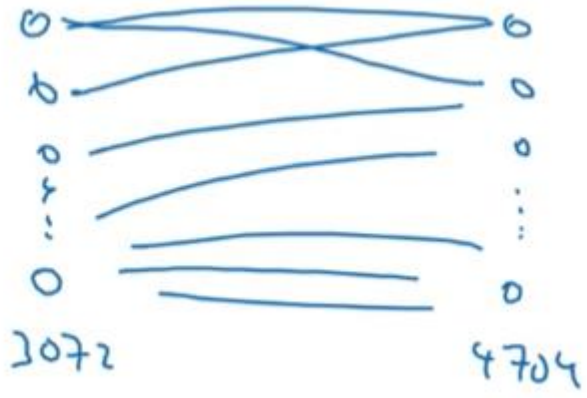
6 x 26 = 156 parameters

Comparing CNN vs ANN

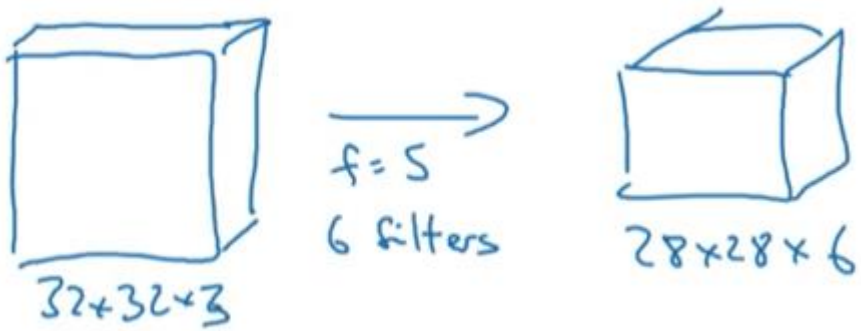


3,072

4,704



Comparing CNN vs ANN



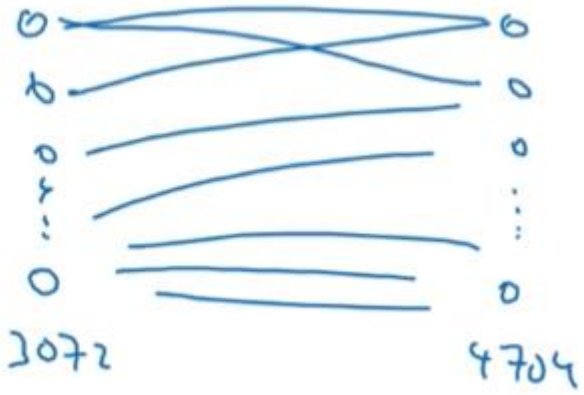
3,072

4,704

$5 \times 5 - 25$

26

$6 \times 26 = 156$ parameters



$3,072 \times 4,704 \approx 14M$

Loss Functions

- Loss functions are the same for MLP
- MSE for regression:

$$L = \frac{1}{2N}\sum_{n=1}^{N}(y_n - \hat{y}_n)^2$$

- Cross Entropy for Classification

$$L = -\sum_{n=1}^{N}\sum_{k=1}^{C}[y_{kn}\log(\hat{y}_{kn})]$$

- Cross Entropy or Dice Coefficient for Image Segmentation

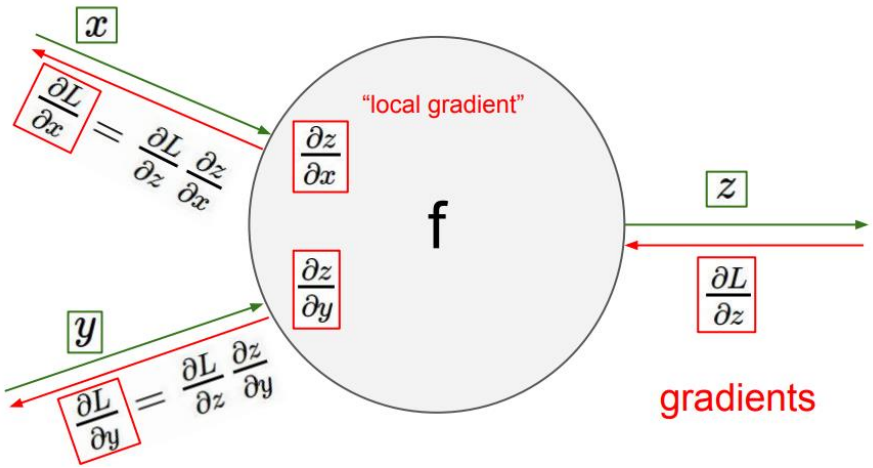
$$L = 1 - \frac{2\sum_{\mathrm{p}} y\hat{y}}{\sum_P y^2 + \sum_P \hat{y}^2}$$

N: number of data samples
C: number of classes
$\hat{y}$: predictions
y: true labels
p: all pixels in an image

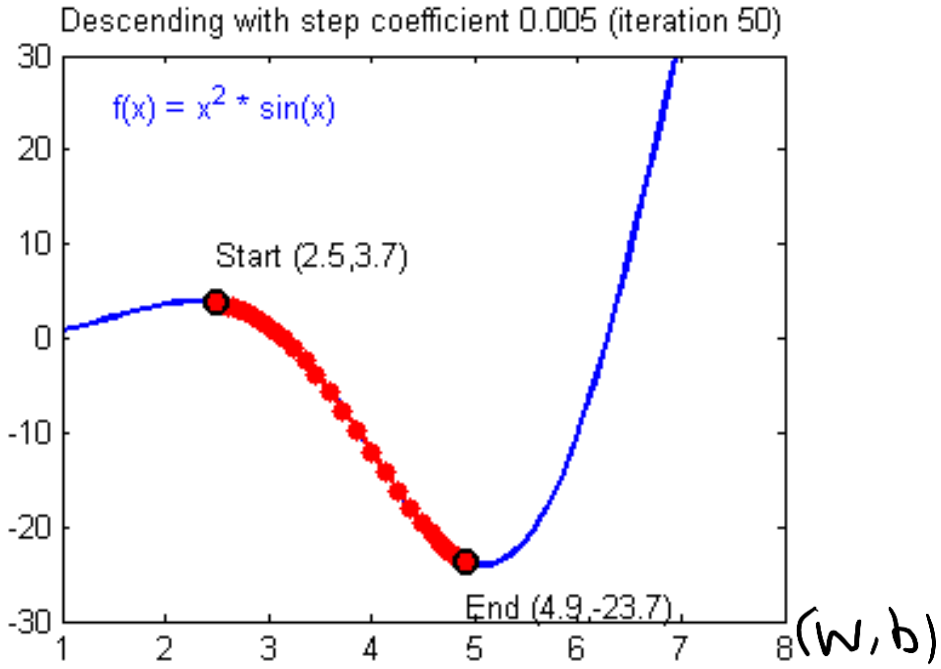
Optimisation

- Optimisation methods: Gradient descent; Gradient descent with momentum; Adam
- Back propagation with chain rule to optimise weight *w* and bias *b*

Chain rule



Gradient Descent



$$W_t = W_{t-1} - \alpha \frac{\partial L}{\partial W_{t-1}}$$

Optimisation Strategy using Big Data

- Normally intrinsic parameters are updated based on the averaged loss value after seeing all the training examples.
  - Too slow to compute when we have a lot of training examples.
- Stochastic Gradient Descent
  - Randomly select ONE training example for gradient calculation.
- Mini Batch Gradient Descent
  - Randomly select a batch of training example for gradient calculation.

Mini-batch Gradient Descent

$$X = [x^{(1)} \; x^{(2)} \; x^{(3)} \; \ldots \; \ldots \; x^{(m)}]$$

(n_x, m)

$$Y = [y^{(1)} \; y^{(2)} \; y^{(3)} \; \ldots \; \ldots \; y^{(m)}]$$

$(1, m)$

What if $m = 5{,}000{,}000$?

Mini-batch Gradient Descent

$$X = [x^{(1)}\ x^{(2)}\ x^{(3)} \cdots \underbrace{x^{(1000)}}][x^{(1001)} \cdots x^{(2000)}] \cdots x^{(m)}]$$

(n_x, m)

$$Y = [y^{(1)}\ y^{(2)}\ y^{(3)} \cdots \cdots y^{(m)}]$$

$(1, m)$

What if $m = 5,000,000$?

Mini-batch Gradient Descent

$X = [x^{(1)}\ x^{(2)}\ x^{(3)} \ldots x^{(1000)}][x^{(1001)} \ldots x^{(2000)}] \ldots x^{(m)}]$

(n_x, m)

X{1} …. X{2} …. X{5000}

$Y = [y^{(1)}\ y^{(2)}\ y^{(3)} \ldots \ldots y^{(m)}]$

$(1, m)$

What if $m = 5{,}000{,}000$?

5,000 mini-batches of 1,000 each

Mini-batch t: $X^{\{t\}}, Y^{\{t\}}$

Mini-batch Gradient Descent

for $t = 1, \ldots, 5000$ {

Forward prop on $X^{\{t\}}$.

$$Z^{[1]} = W^{[1]} X^{\{t\}} + b^{[1]}$$
$$A^{[1]} = g^{[1]}(Z^{[1]})$$
$$\vdots$$
$$A^{[l]} = g^{[l]}(Z^{[l]})$$

Mini-batch Gradient Descent

for $t = 1, \ldots, 5000$ {

Forward prop on $X^{\{t\}}$.

$$Z^{[1]} = W^{[1]} X^{\{t\}} + b^{[1]}$$
$$A^{[1]} = g^{[1]}(Z^{[1]})$$
$$\vdots$$
$$A^{[L]} = g^{[L]}(Z^{[L]})$$

}

1 step of gradient dest using $X^{\{t\}}, Y^{\{t\}}$.
(as if $m = 1000$)

X, Y

Mini-batch Gradient Descent

1 step of gradient dest using $X^{\{t\}}, Y^{\{t\}}$.
(as if $m = 1000$)

X, Y

for $t = 1, \ldots, 5000$ {

Forward prop on $X^{\{t\}}$.

$$Z^{[1]} = W^{[1]} X^{\{t\}} + b^{[1]}$$
$$A^{[1]} = g^{[1]}(Z^{[1]})$$
$$\vdots$$
$$A^{[L]} = g^{[L]}(Z^{[L]})$$

Compute cost $J^{\{t\}} = \frac{1}{1000} \sum_{i=1}^{l} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$ from $X^{\{t\}}, Y^{\{t\}}$.

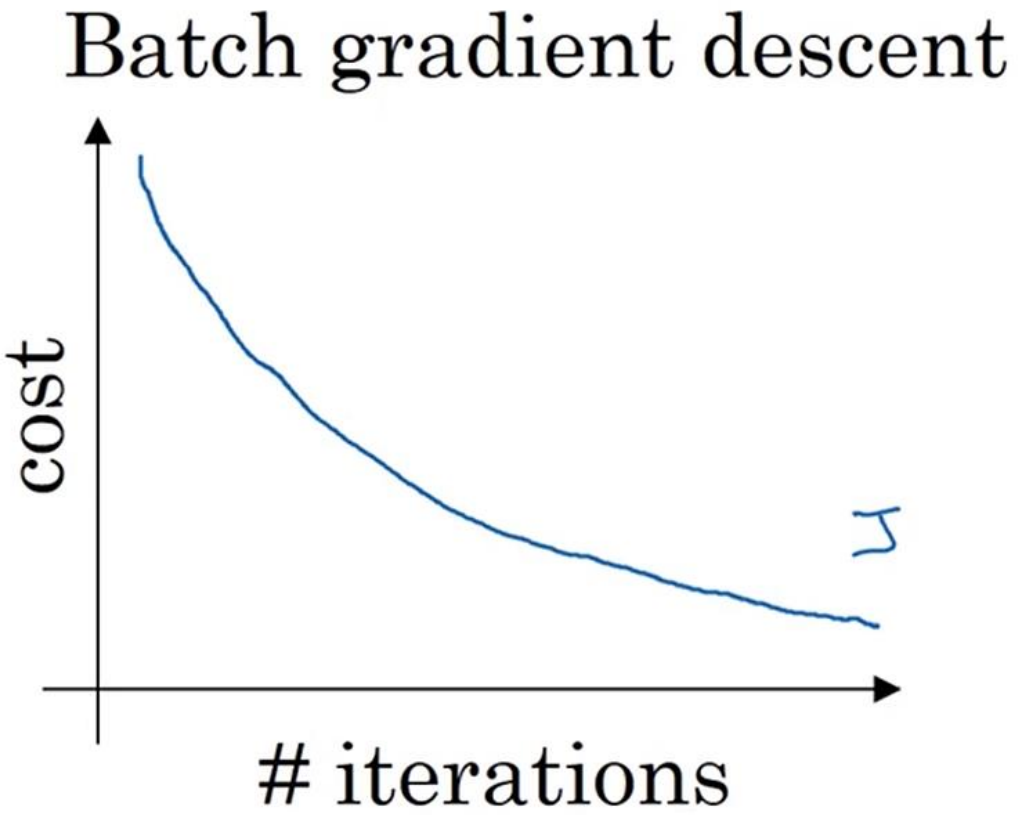
Backprop to compute gradients wrt $J^{\{t\}}$ (using $(X^{\{t\}}, Y^{\{t\}})$)

$W^{[l]} := W^{[l]} - \alpha dW^{[l]}$, $b^{[l]} := b^{[l]} - \alpha db^{[l]}$

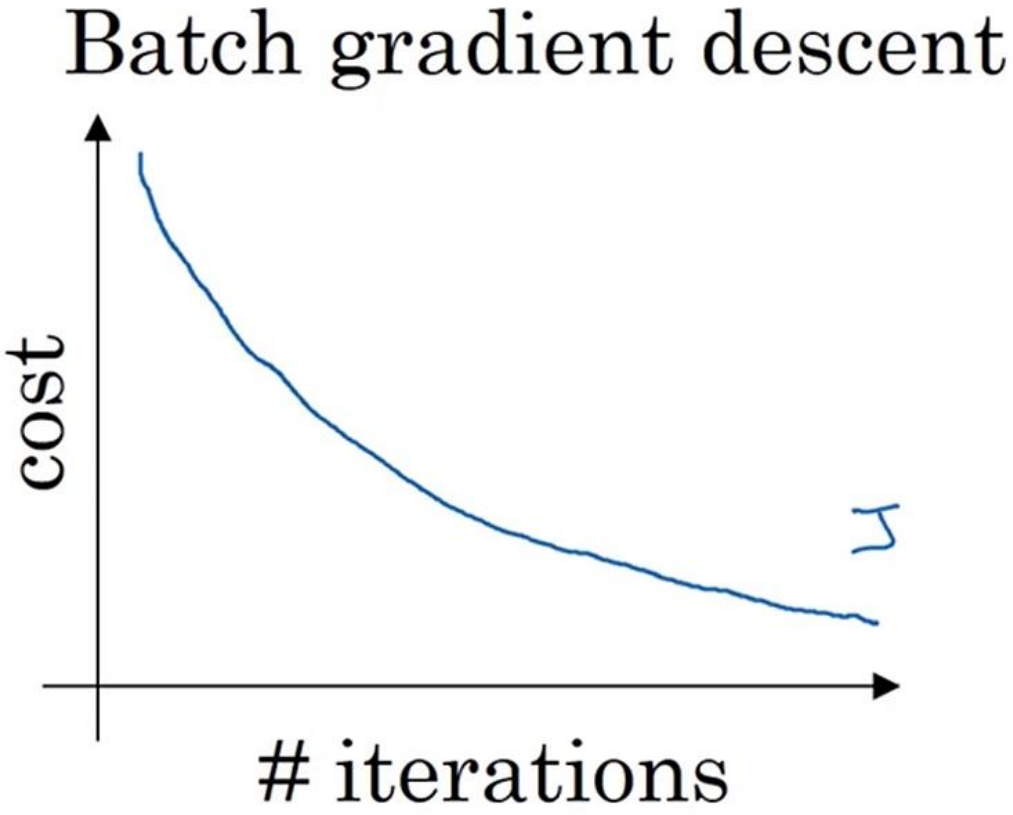
}

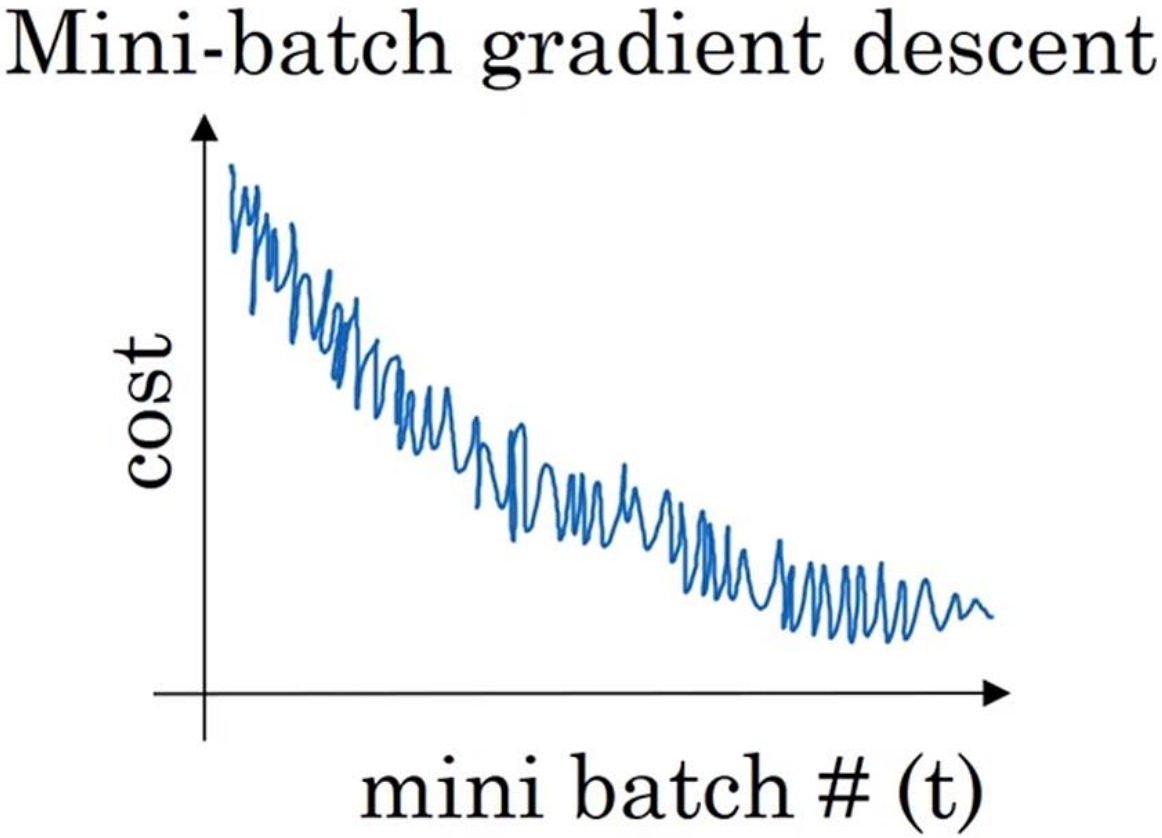
"1 epoch"
└ pass through training set.

Mini-batch Gradient Descent



Mini-batch Gradient Descent



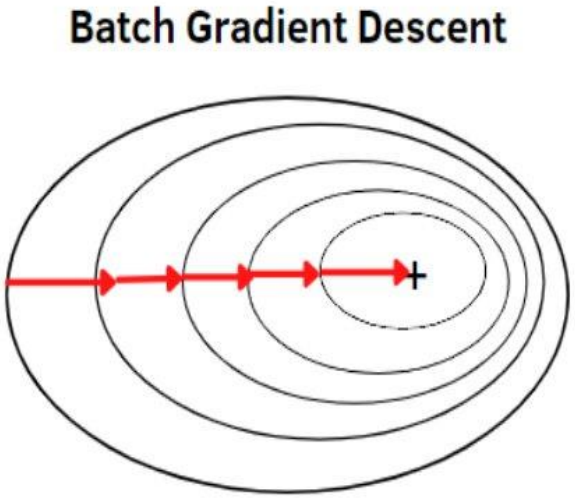


Mini-batch size

	#Samples	#Iterations
Batch Gradient Descent	The entire Dataset	1
Mini-Batch Gradient Descent	Subsets of the dataset	$\frac{N}{\text{size of mini-batch}}$
Stochastic Gradient Descent	Each Sample of the dataset	N

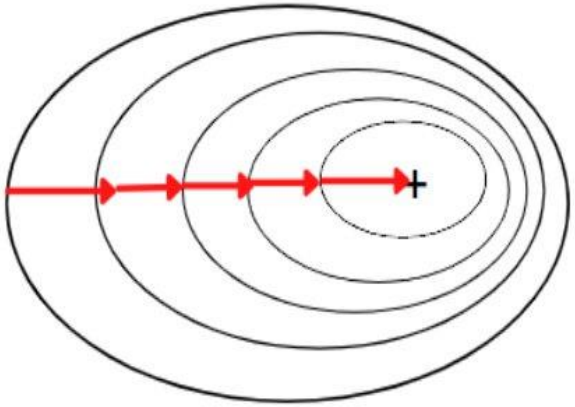
Rule of thumb: batch size is 2^x due to how memory is stored

SGD vs BGD vs MBGD

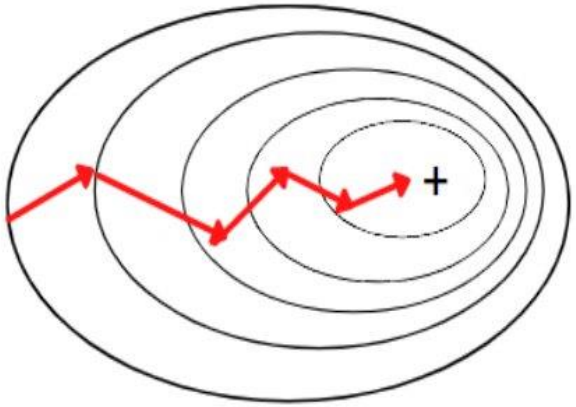


SGD vs BGD vs MBGD

Batch Gradient Descent

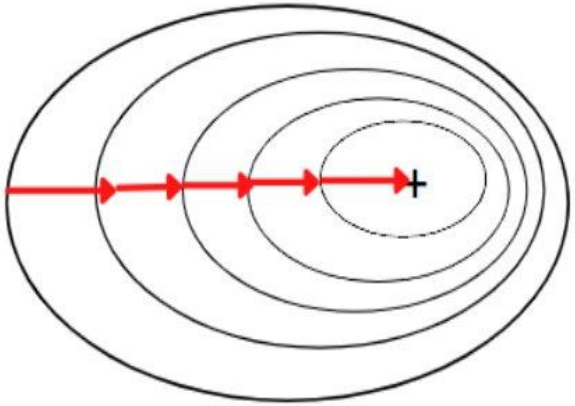


Mini-Batch Gradient Descent

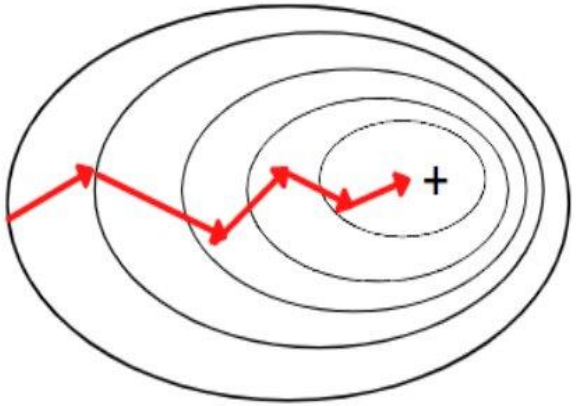


SGD vs BGD vs MBGD

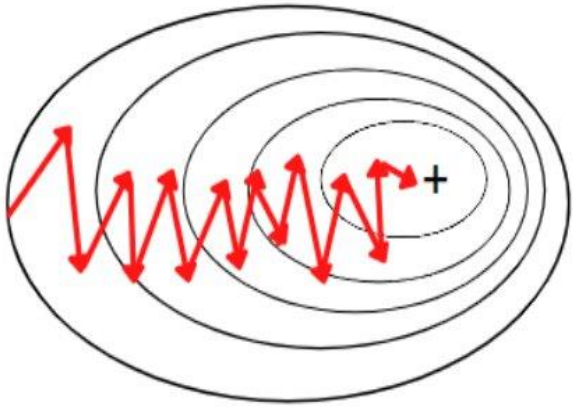
Batch Gradient Descent



Mini-Batch Gradient Descent

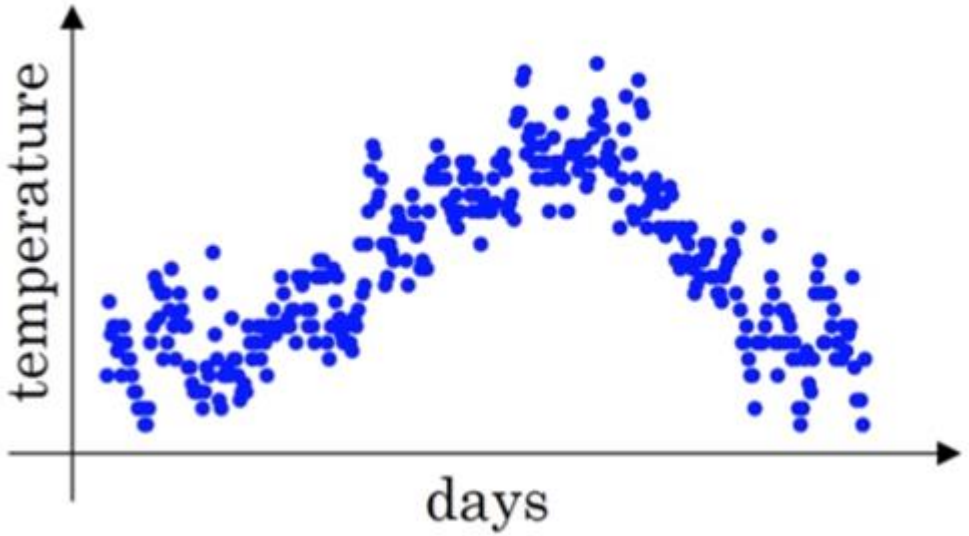


Stochastic Gradient Descent



Exponential Moving Average

- Emphasizes recent data
- Responsive to trends
- Reduces noise



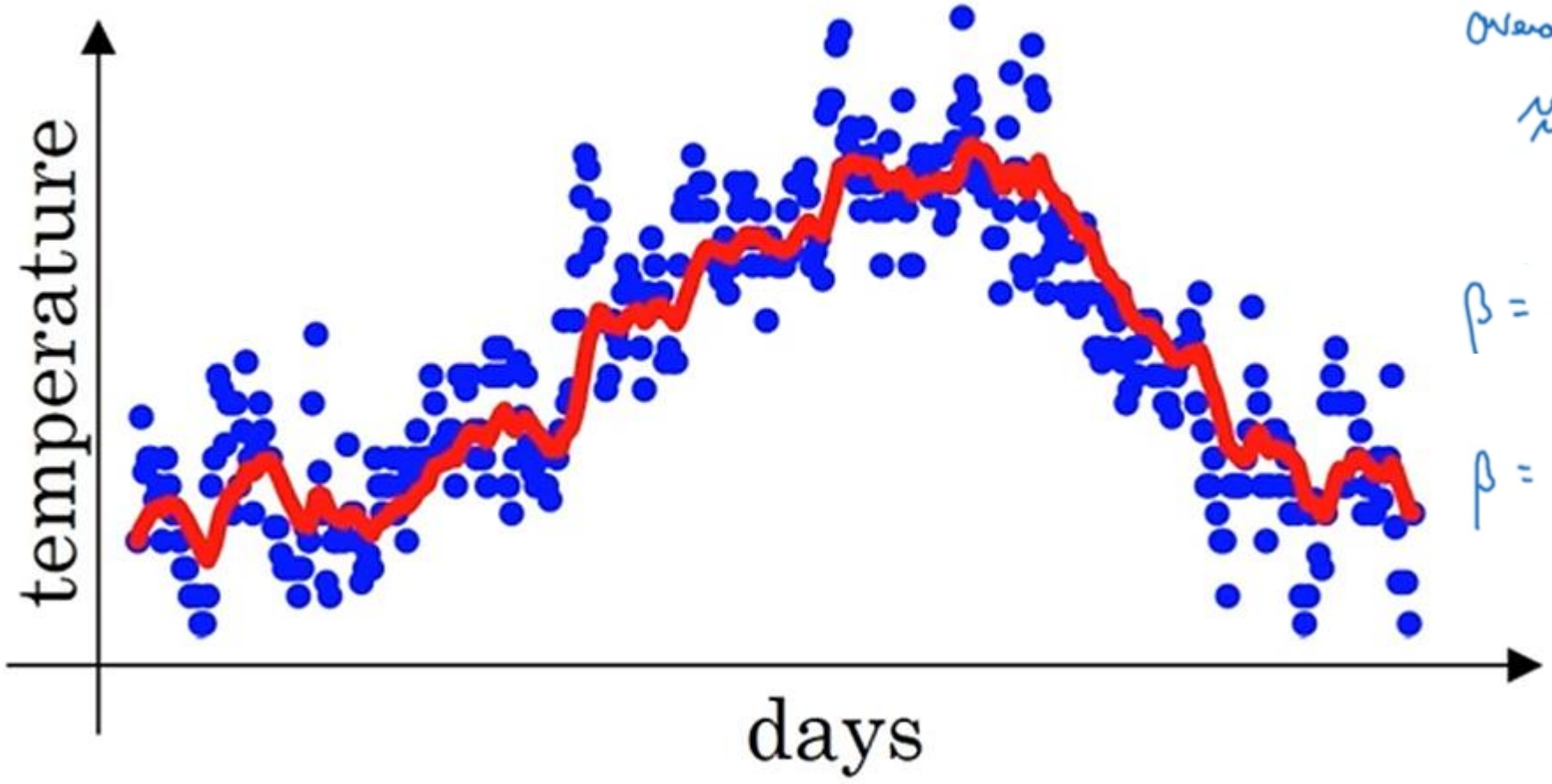
Exponential Averaging

$$V_t = \beta V_{t-1} + (1-\beta)\theta_t$$

Average over $\approx \frac{1}{1-\beta}$ days

$\beta = 0.9$: ≈ 10 days

$\beta = 0.98$: ≈ 50 days



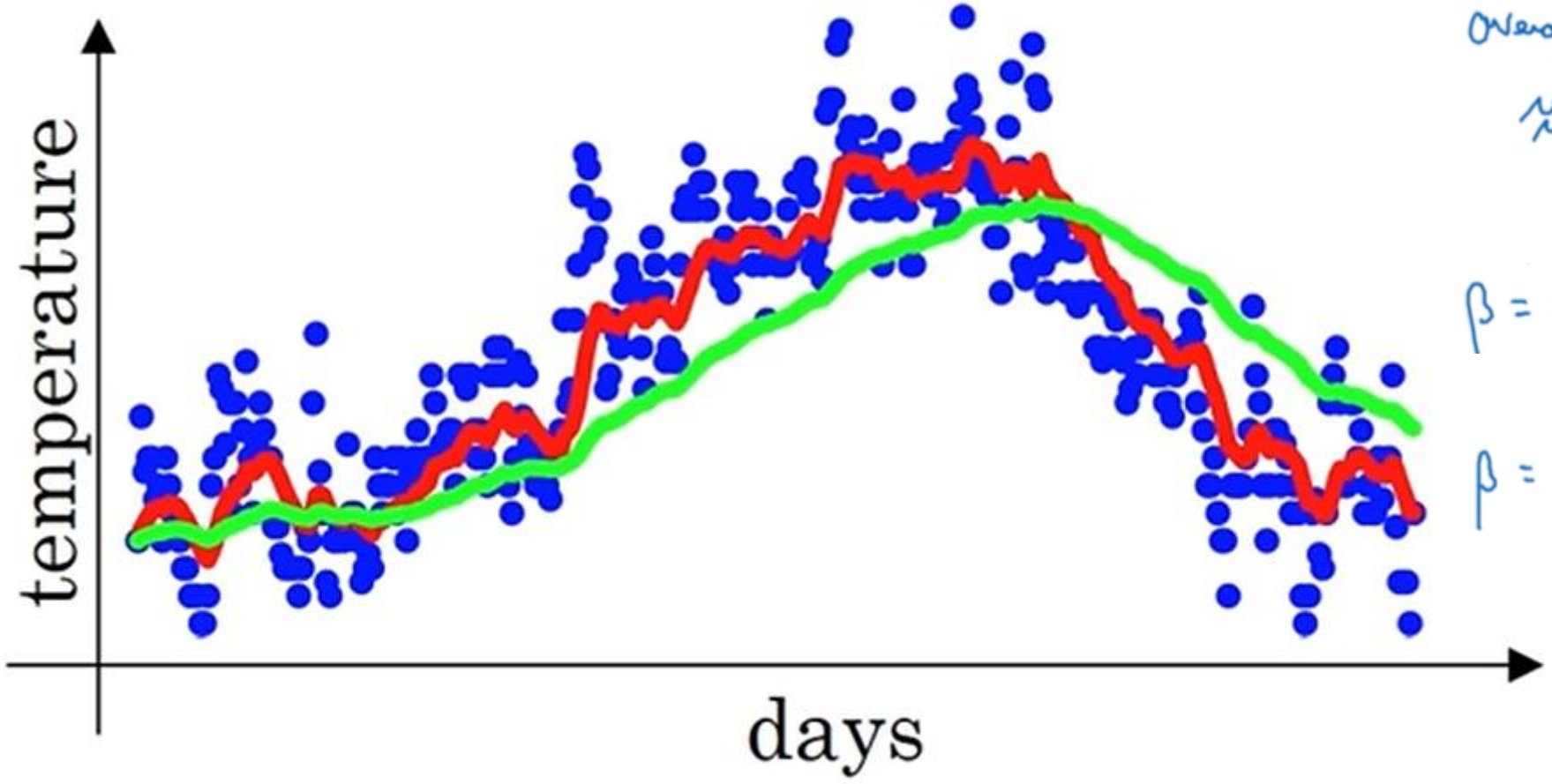
Exponential Averaging

$$V_t = \beta V_{t-1} + (1-\beta)\theta_t$$

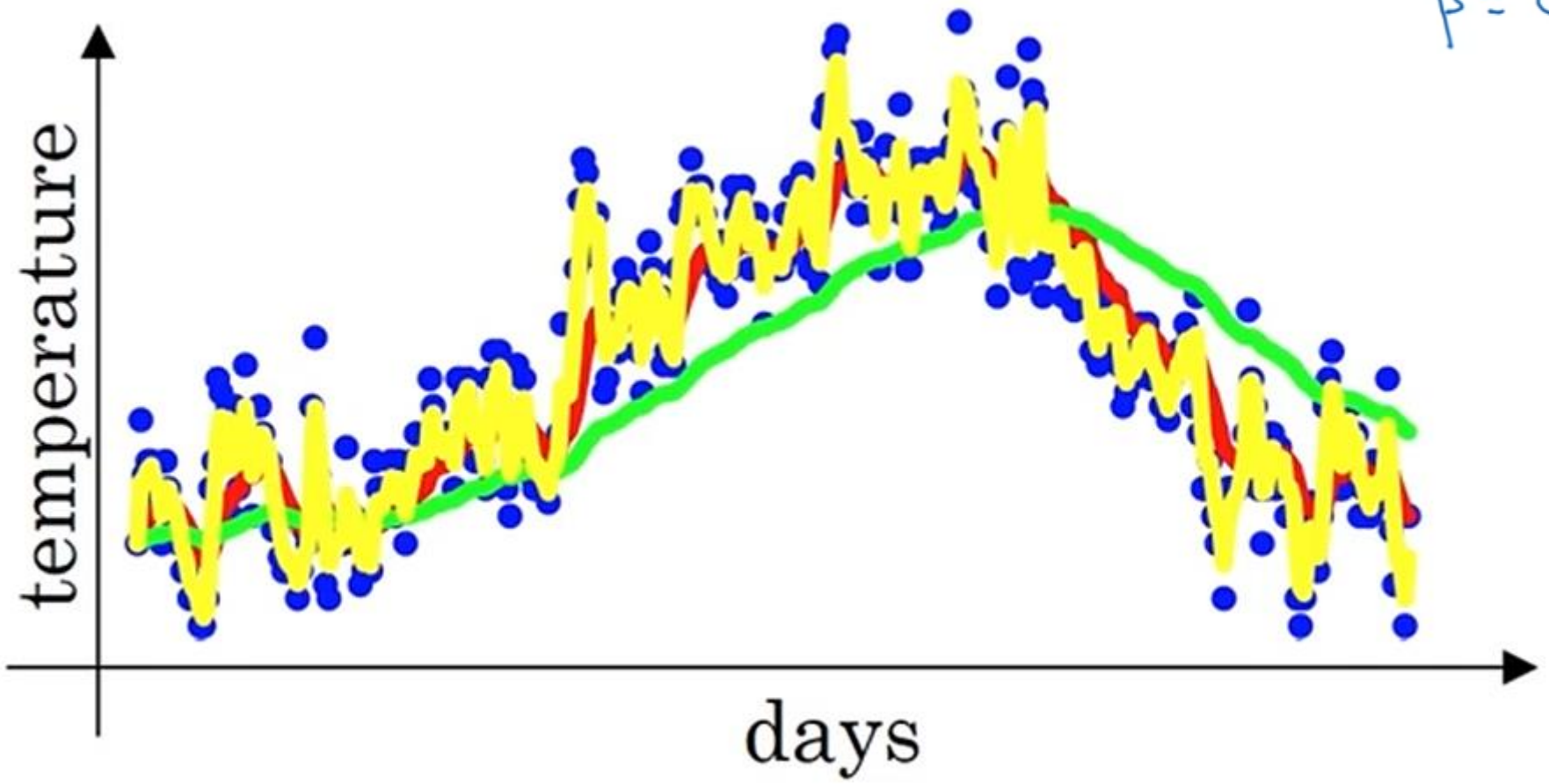
Average over $\approx \frac{1}{1-\beta}$ days

$\beta = 0.9$: ≈ 10 days

$\beta = 0.98$: ≈ 50 days



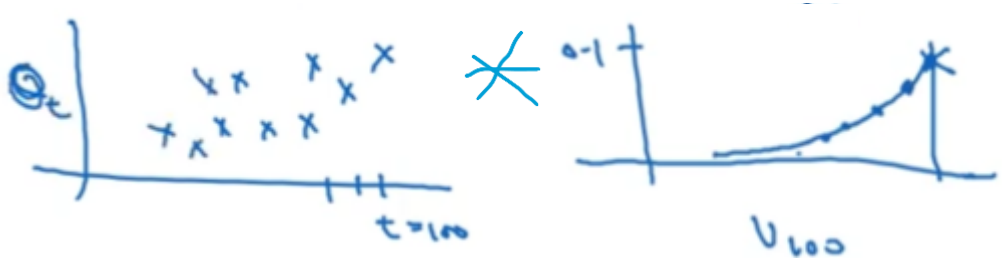
Exponential Averaging



$\beta = 0.5$: ≈ 2 days

Exponential averaging

$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$



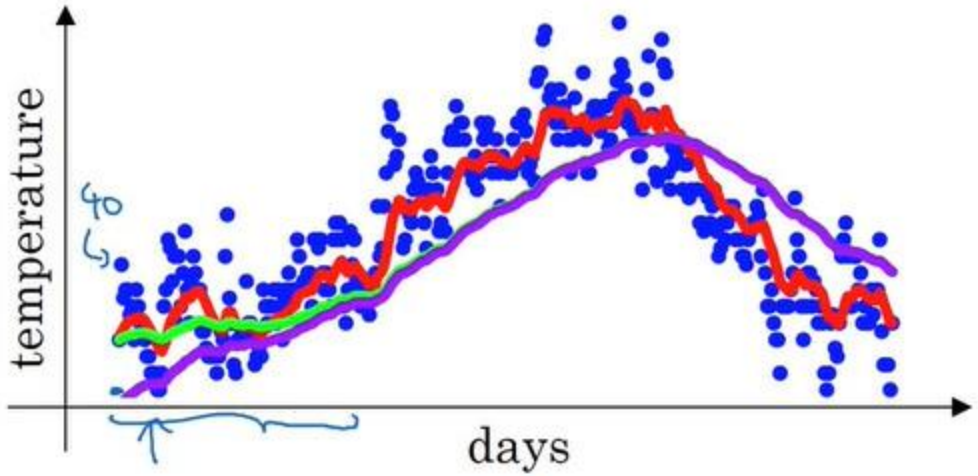
$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$
$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$
$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$

$$V_{100} = 0.1\,\theta_{100} + 0.9\,V_{99}$$

$$V_{100} = 0.1\,\theta_{100} + 0.9\,(0.1\,\theta_{99} + 0.9\,V_{98})$$

$$= 0.1\,\theta_{100} + 0.1 \times 0.9 \cdot \theta_{99} + 0.1\,(0.9)^2\,\theta_{98} + 0.1\,(0.9)^3\,\theta_{97} + 0.1\,(0.9)^4\,\theta_{96} + \cdots$$

Bias correction in EMA



$\beta = 0.98$

$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

$V_0 = 0$

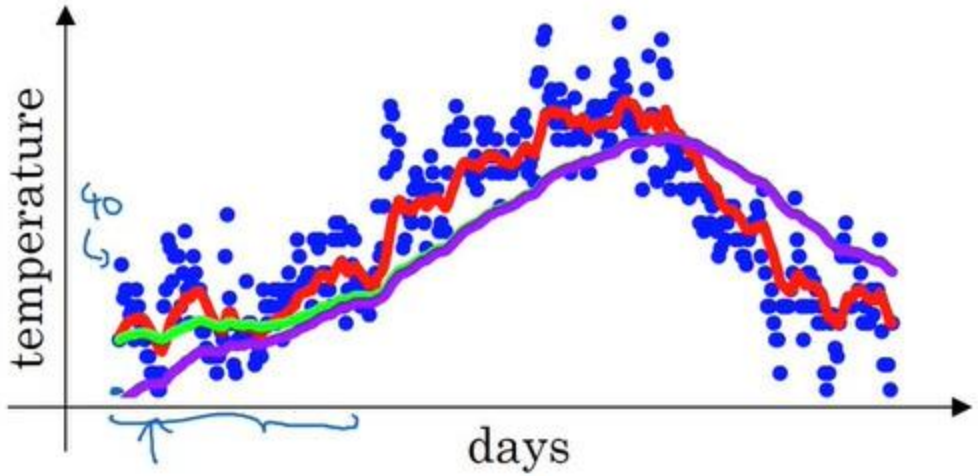
$V_1 = 0.98 V_0 + 0.02\,\theta_1$

$V_2 = 0.98\,V_1 + 0.02\,\theta_2$

$= 0.98 \times 0.02 \times \theta_1 + 0.02\,\theta_2$

$= 0.0196\,\theta_1 + 0.02\,\theta_2$

Bias correction in EMA



$\beta = 0.98$

$$\rightarrow v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

$V_0 = 0$

$V_1 = 0.98 V_0 + 0.02\,\theta_1$

$V_2 = 0.98\,V_1 + 0.02\,\theta_2$

$= 0.98 \times 0.02 \times \theta_1 + 0.02\,\theta_2$

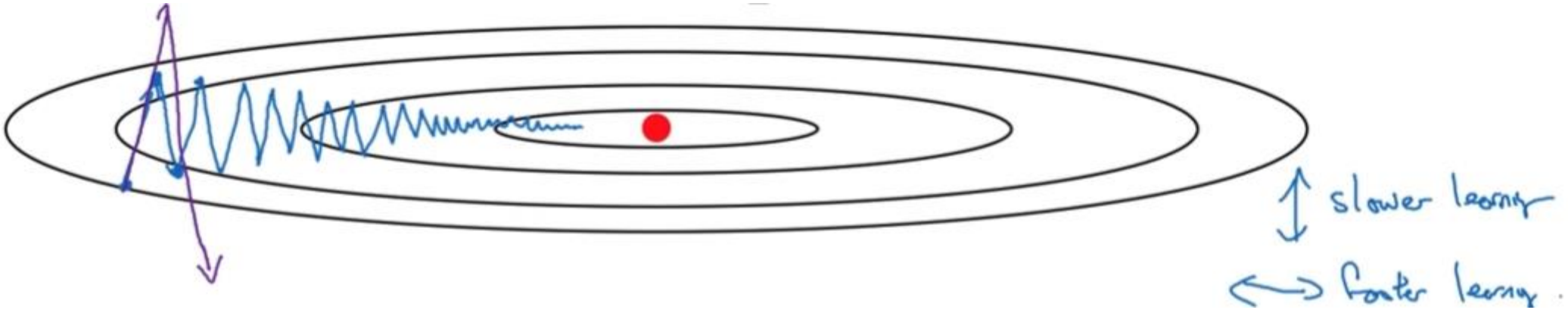
$= 0.0196\,\theta_1 + 0.02\,\theta_2$

$$\frac{V_t}{1-\beta^t}$$

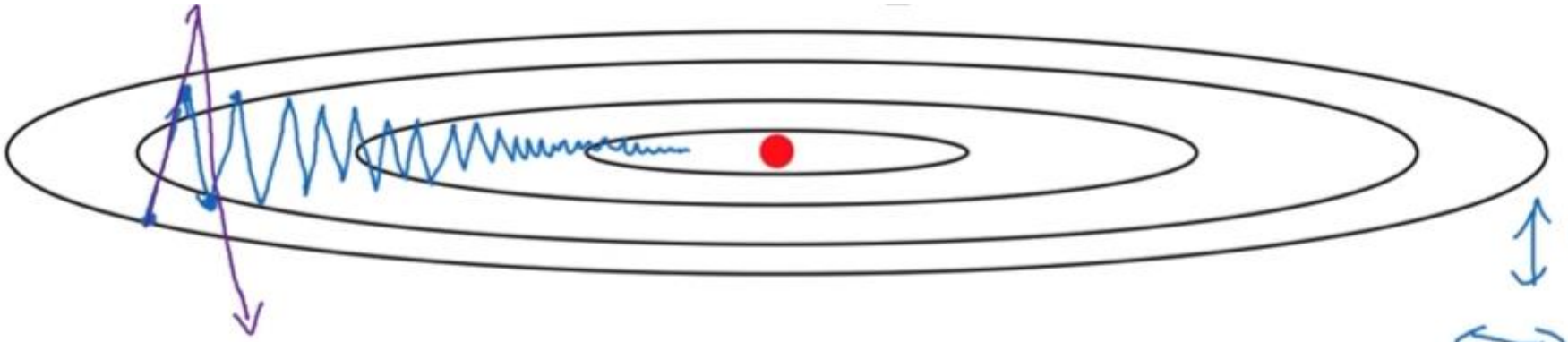
$t=2: \quad 1-\beta^t = 1-(0.98)^2 = 0.0396$

$$\frac{V_2}{0.0396} = \frac{0.0196\theta_1 + 0.02\theta_2}{0.0396}$$

Optimisation: gradient descent with momentum



Optimisation: gradient descent with momentum



Momentum:

On iteration t:

Compute dW, db on current mini-batch.

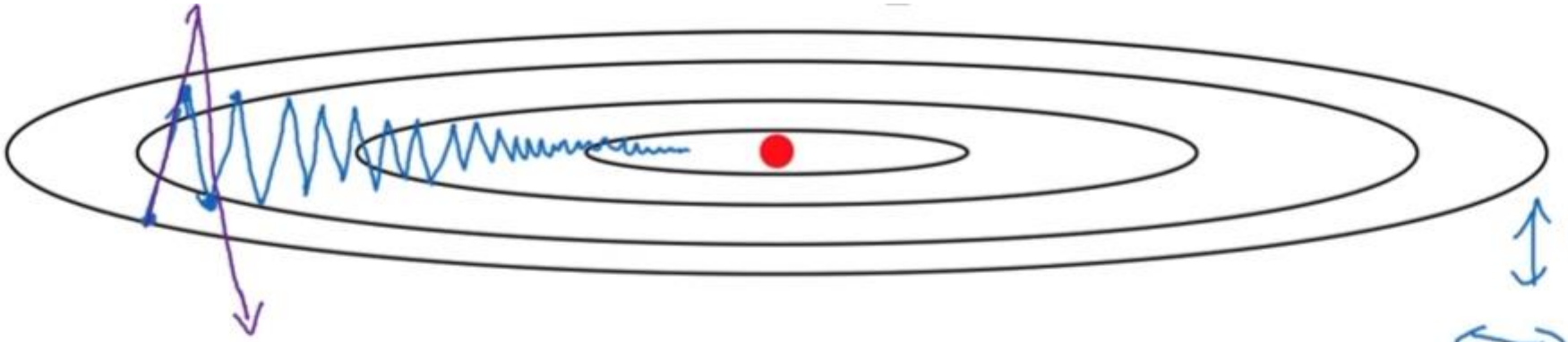
$$v_{dW} = \beta v_{dW} + (1-\beta) dW$$

$$v_{db} = \beta v_{db} + (1-\beta) db$$

"$v_\theta = \beta v_{\theta-1} + (1-\beta)\theta_t$"

$$W := W - \alpha v_{dW}, \quad b := b - \alpha v_{db}$$

Optimisation: gradient descent with momentum



Momentum:

On iteration t:

Compute dW, db on current mini-batch.

$$v_{dW} = \beta v_{dW} + (1-\beta) dW$$

$$v_{db} = \beta v_{db} + (1-\beta) db$$

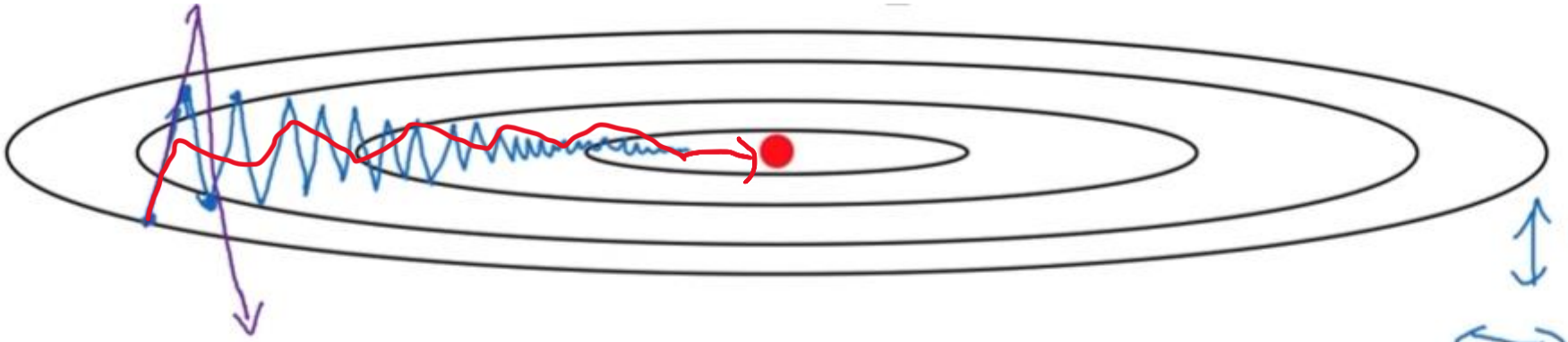
"$v_\theta = \beta v_{\theta-1} + (1-\beta)\theta_t$"

$$W := W - \alpha v_{dW}, \quad b := b - \alpha v_{db}$$

Velocity

Acceleration

Optimisation: gradient descent with momentum



Momentum:

On iteration t:

Compute dW, db on current mini-batch.

$$v_{dW} = \beta v_{dW} + (1-\beta) dW$$

$$v_{db} = \beta v_{db} + (1-\beta) db$$

"$v_\theta = \beta v_{\theta-1} + (1-\beta)\theta_t$"

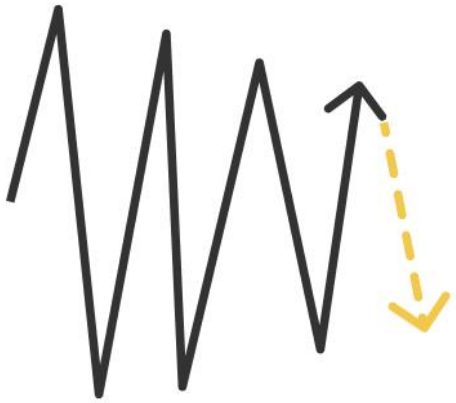
$$W := W - \alpha v_{dW}, \quad b := b - \alpha v_{db}$$

Velocity

Acceleration

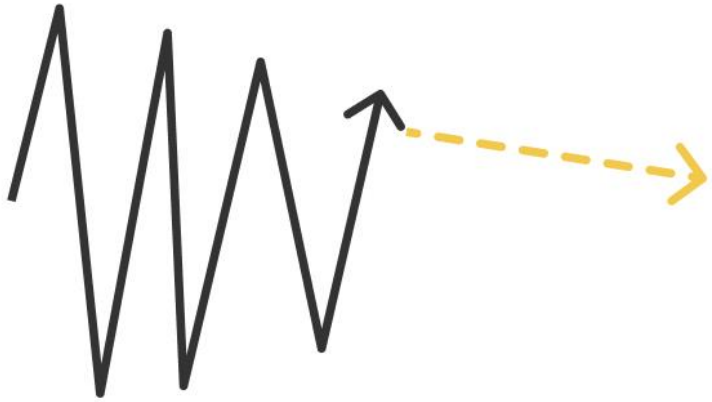
Optimisation: gradient descent with momentum

Gradient descent



the gradient computed on the current iteration does not prevent gradient descent from oscillating in the vertical direction

Momentum



the average vector of the horizontal component is aligned towards the minimum

the average vector of the vertical component is close to 0

Optimisation: gradient descent with momentum

Algorithm:

On iteration t:

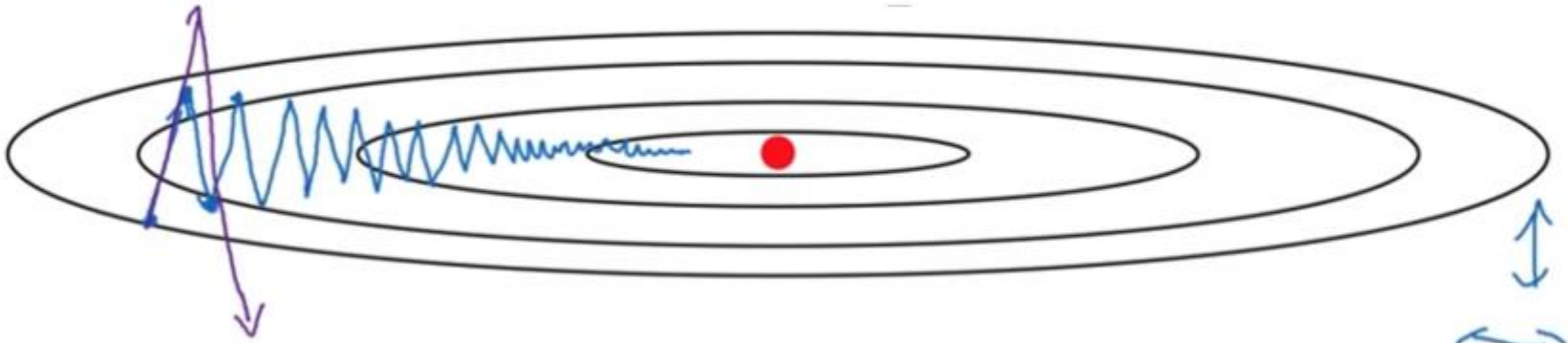
Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

Optimisation: RMSProp



On iteration t:

Compute dW, db on current mini-batch

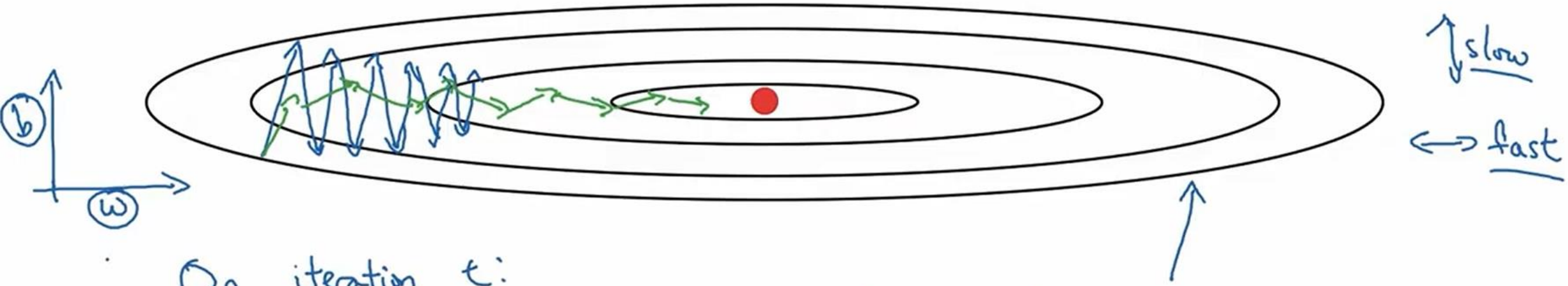
$S_{dW} = \beta S_{dW} + (1-\beta) \underline{dW^2}$ ← element-wise ← small

$S_{db} = \beta S_{db} + (1-\beta) db^2$ ← large

$W := W - \alpha \frac{dW}{\sqrt{S_{dW}}}$ ←

$b := b - \alpha \frac{db}{\sqrt{S_{db}}}$ ←

Optimisation: RMSProp



On iteration t:

Compute dW, db on current mini-batch

$S_{dW} = \beta S_{dW} + (1-\beta) \underline{dW^2}$ ← element-wise ← small

$S_{db} = \beta S_{db} + (1-\beta) db^2$ ← large

$W := W - \alpha \frac{dW}{\sqrt{S_{dW}}}$ ←

$b := b - \alpha \frac{db}{\sqrt{S_{db}}}$ ←

Optimisation: RMSProp

RMSProp: adaptive learning rate for each parameter

$$v_{dw} = \beta \cdot v_{dw} + (1 - \beta) \cdot dw^2$$

$$v_{db} = \beta \cdot v_{dw} + (1 - \beta) \cdot db^2$$

$$W = W - \alpha \cdot \frac{dw}{\sqrt{v_{dw}} + \epsilon}$$

$$b = b - \alpha \cdot \frac{db}{\sqrt{v_{db}} + \epsilon}$$

Optimisation: Adam

$V_{dw} = 0$, $S_{dw} = 0$. $V_{db} = 0$, $S_{db} = 0$

On iteration t:

Compute dW, db using current mini-batch

$V_{dw} = \beta_1 V_{dw} + (1-\beta_1) dW$, $V_{db} = \beta_1 V_{db} + (1-\beta_1) db$

$S_{dw} = \beta_2 S_{dw} + (1-\beta_2) dW^2$, $S_{db} = \beta_2 S_{db} + (1-\beta_2) db$

Optimisation: Adam

$V_{dW} = 0$, $S_{dW} = 0$. $V_{db} = 0$, $S_{db} = 0$

On iteration t:

Compute dW, db using current mini-batch

$V_{dW} = \beta_1 V_{dW} + (1-\beta_1) dW$, $V_{db} = \beta_1 V_{db} + (1-\beta_1) db$ ← "momentum" β_1

$S_{dW} = \beta_2 S_{dW} + (1-\beta_2) dW^2$, $S_{db} = \beta_2 S_{db} + (1-\beta_2) db$ ← "RMSprop" β_2

Optimisation: Adam

$V_{dW} = 0$, $S_{dW} = 0$. $V_{db} = 0$, $S_{db} = 0$

On iteration t:

Compute dW, db using current mini-batch

$V_{dW} = \beta_1 V_{dW} + (1-\beta_1) dW$, $V_{db} = \beta_1 V_{db} + (1-\beta_1) db$ ← "momentum" β_1

$S_{dW} = \beta_2 S_{dW} + (1-\beta_2) dW^2$, $S_{db} = \beta_2 S_{db} + (1-\beta_2) db$ ← "RMSprop" β_2

$V_{dW}^{corrected} = V_{dW} / (1-\beta_1^t)$, $V_{db}^{corrected} = V_{db} / (1-\beta_1^t)$

$S_{dW}^{corrected} = S_{dW} / (1-\beta_2^t)$, $S_{db}^{corrected} = S_{db} / (1-\beta_2^t)$

$$W := W - \alpha \frac{V_{dW}^{corrected}}{\sqrt{S_{dW}^{corrected}} + \varepsilon} \qquad b := b - \alpha \frac{V_{db}^{corrected}}{\sqrt{S_{db}^{corrected}} + \varepsilon}$$

Optimisation: Adam (Adaptive Moment Estimation)

- Adam is the most popular optimisation method used in deep learning: combines RMSProp and Momentum

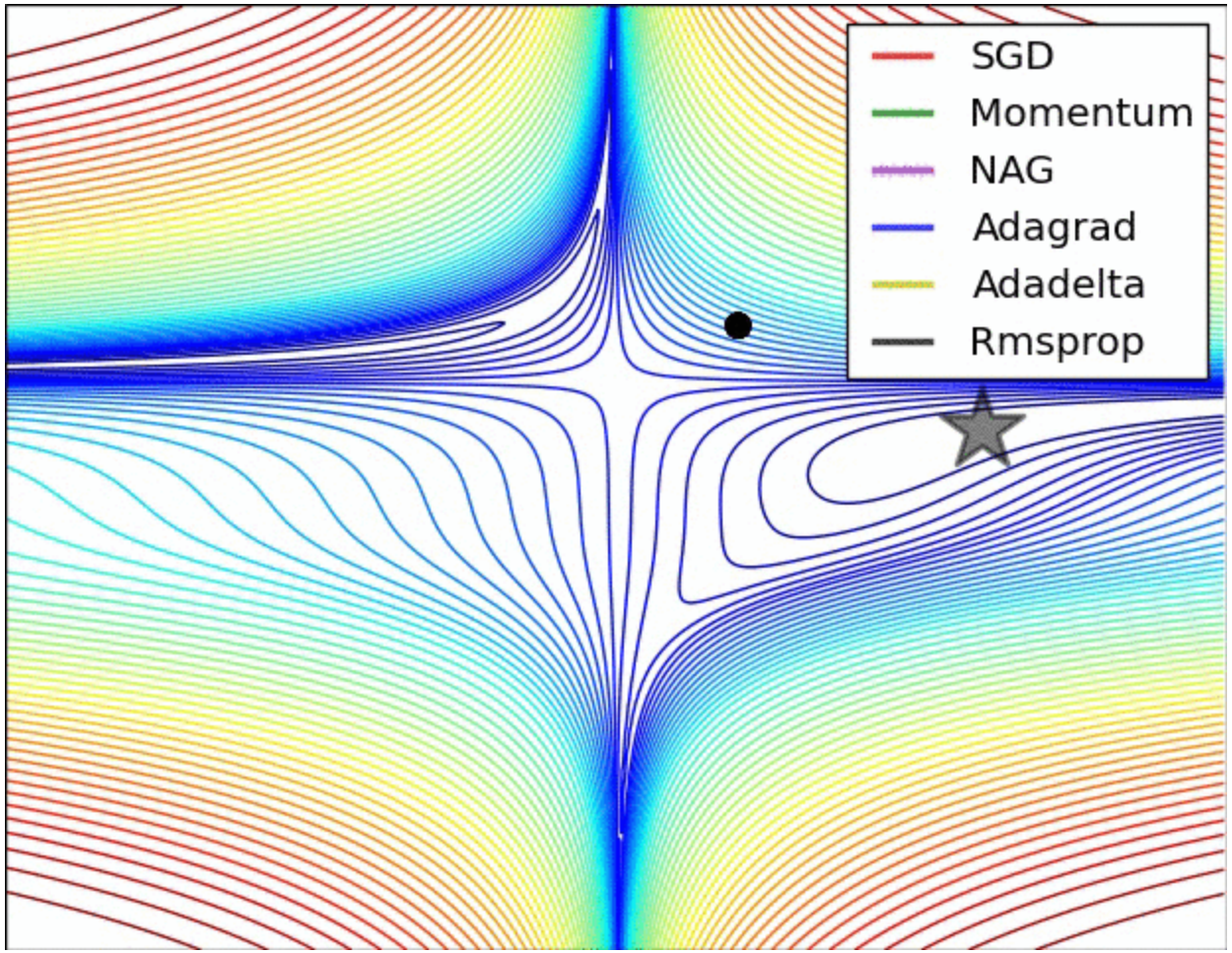
$$w_t = w_{t-1} - \alpha \frac{v_w}{\sqrt{s_w + \varepsilon}}$$

$$v_w = \beta_1 v_{w-1} + (1 - \beta_1) g_{w_{t-1}}$$

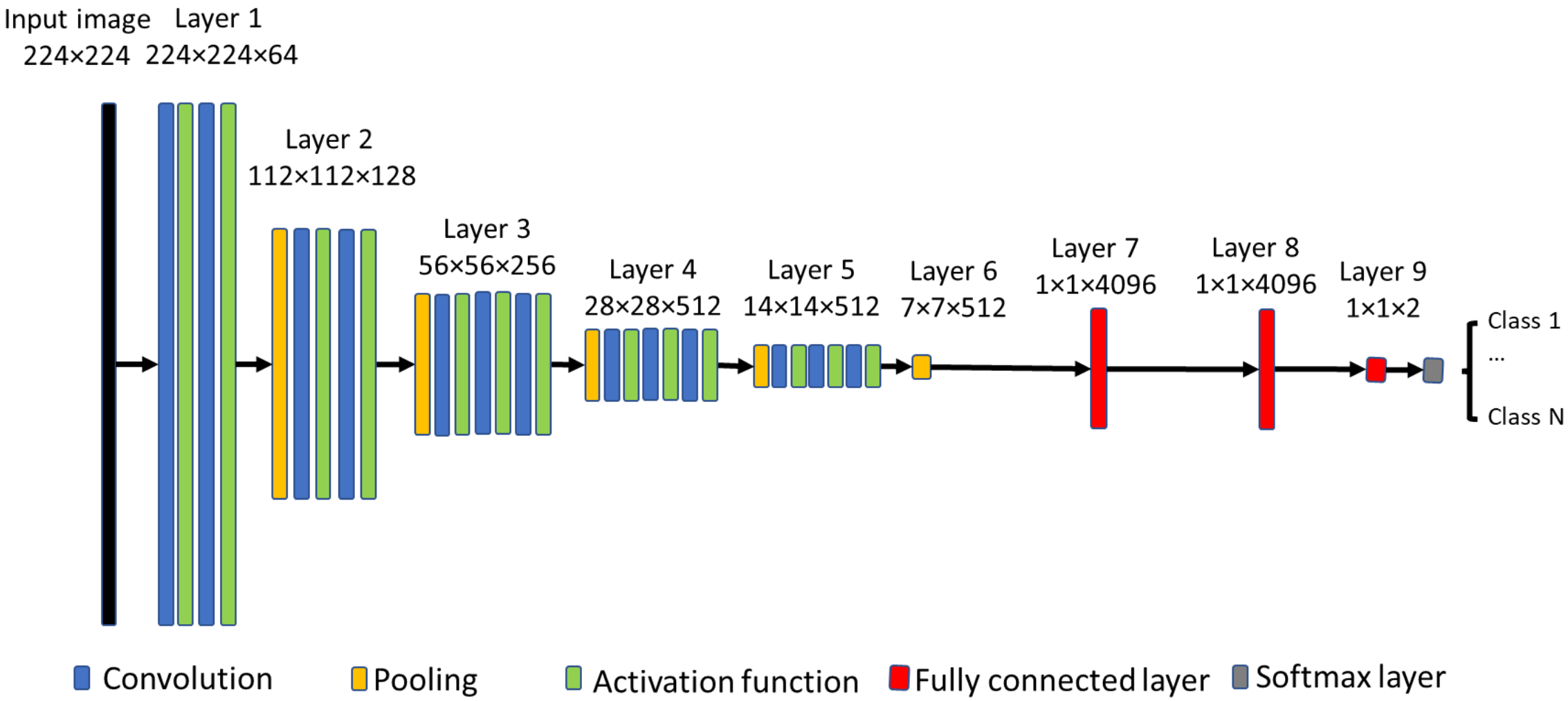
$$s_w = \beta_2 s_{w_{-1}} + (1 - \beta_2) g^2_{w_{t-1}}$$

α : needs to be tuned
β_1: 0.9
β_2: 0.999
ε: 10^{-8}

Comparing optimisers

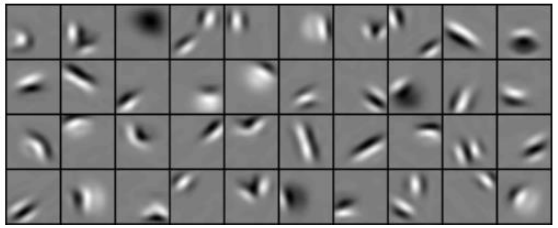


Overview

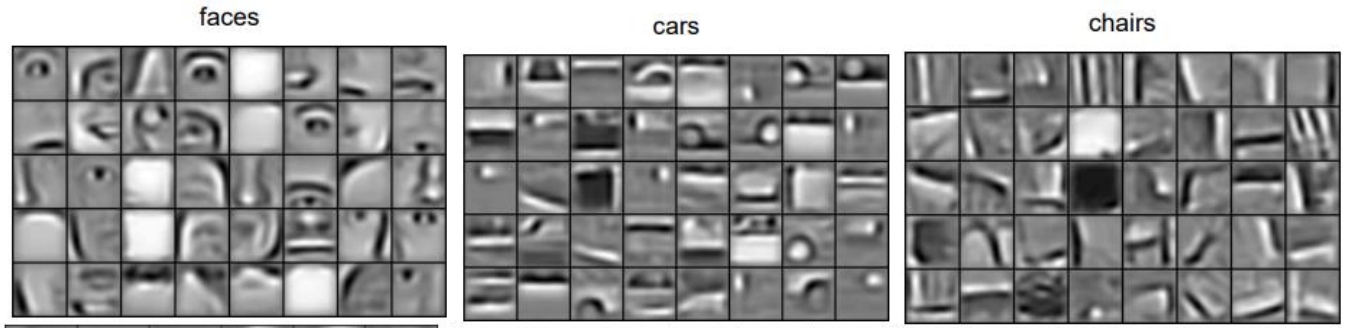

Input image
224×224
Layer 1
224×224×64
Layer 2
112×112×128
Layer 3
56×56×256
Layer 4
28×28×512
Layer 5
14×14×512
Layer 6
7×7×512
Layer 7
1×1×4096
Layer 8
1×1×4096
Layer 9
1×1×2
Class 1
...
Class N
Convolution
Pooling
Activation function
Fully connected layer
Softmax layer

What did the network learn?

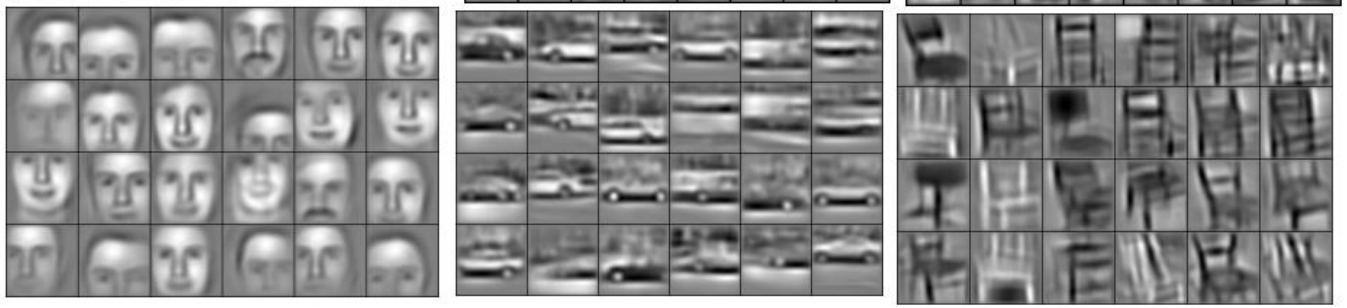
Low-level filters



Mid-level filters



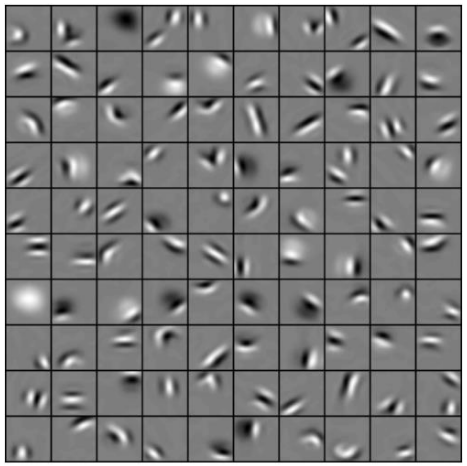
High-level filters



Network Visualisation

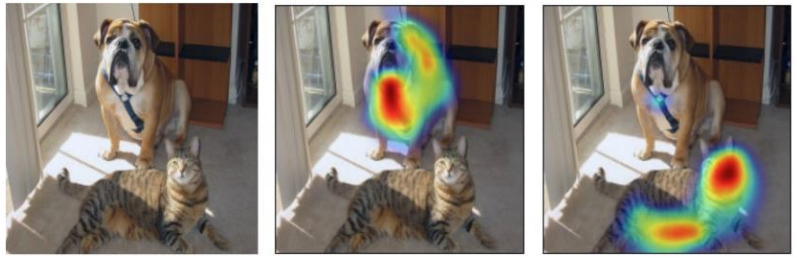
- Visualise the filters [1]
- Visualise the feature maps
- Generate activation maps [2,3]

Learned filters



Feature maps

Grad CAM



[1] H. Lee, et al., Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations, 2009
[2] K Simonyan, et al., Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps, 2014
[3] R. Selvaraju, et al., Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization, 2016

What will we learn in the next lecture?

- Data Augmentation
- Regularization
- Dropout
- Transfer Learning
- Popular CNNs
- Image Segmentation
- Popular Applications